

Unsupervised Learning :

What is Unsupervised Learning?

As the name suggests, unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things. It can be defined as:

Unsupervised learning is a type of machine learning in which models are trained using unlabelled dataset and are allowed to act on that data without any supervision.

Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format.**

Example: Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset. The task of the unsupervised learning algorithm is to identify the image features on their own. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.



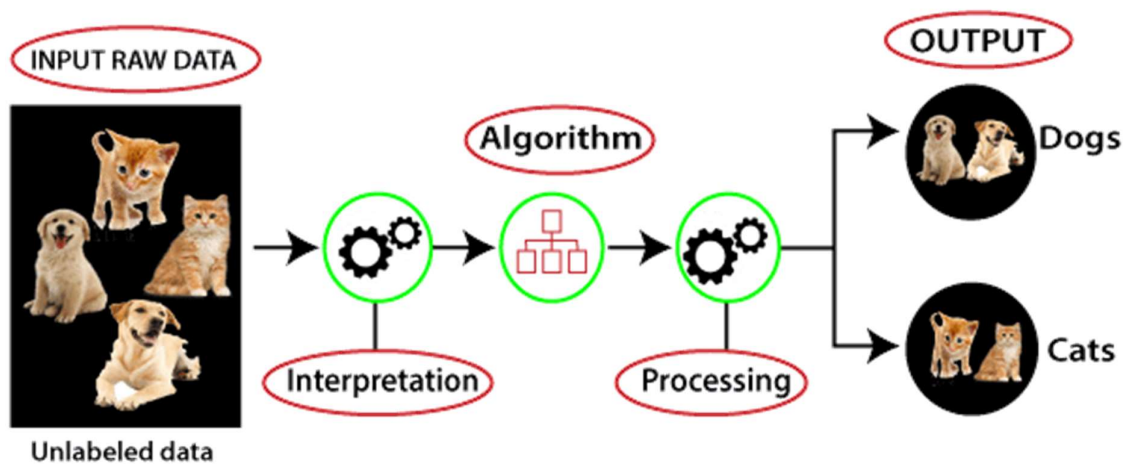
Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabelled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

Working of Unsupervised Learning

Working of unsupervised learning can be understood by the below diagram:

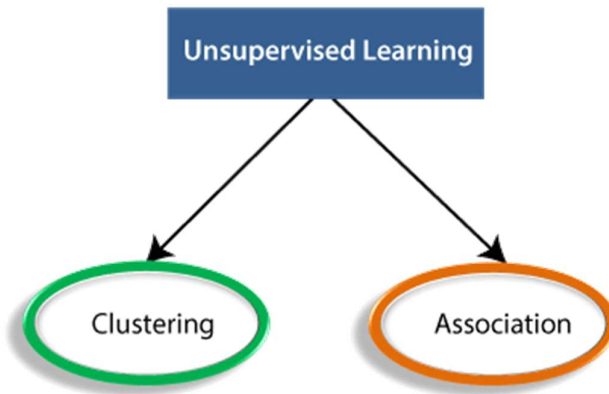


Here, we have taken an unlabelled input data, which means it is not categorized and corresponding outputs are also not given. Now, this unlabelled input data is fed to the machine learning model in order to train it. Firstly, it will interpret the raw data to find the hidden patterns from the data and then will apply suitable algorithms such as k-means clustering, Decision tree, etc.

Once it applies the suitable algorithm, the algorithm divides the data objects into groups according to the similarities and difference between the objects.

Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:



- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remain in a group and have less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.
- **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

Note: We will learn these algorithms in later chapters.

Unsupervised Learning algorithms:

Below is the list of some popular unsupervised learning algorithms:

- **K-means clustering**
- **Hierarchical clustering**

K-Means Clustering Algorithm

K-Means Clustering is an **Unsupervised Learning algorithm**, which groups the unlabelled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on.

It is an iterative algorithm that divides the unlabelled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.

It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.

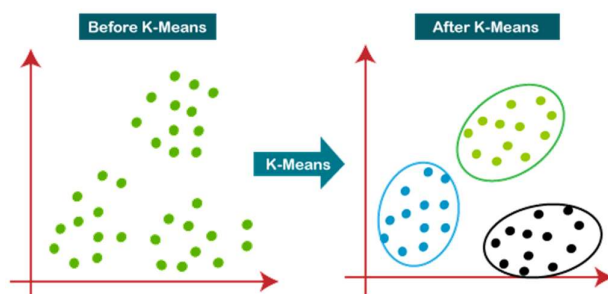
The algorithm takes the unlabelled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm.

The k-means **clustering** algorithm mainly performs two tasks:

- Determines the best value for K centre points or centroids by an iterative process.
- Assigns each data point to its closest k-centre. Those data points which are near to the particular k-centre, create a cluster.

Hence each cluster has datapoints with some commonalities, and it is away from other clusters.

The below diagram explains the working of the K-means Clustering Algorithm:



How does the K-Means Algorithm Work?

The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

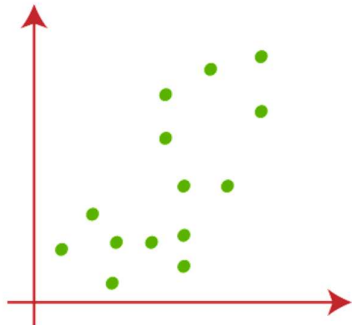
Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.

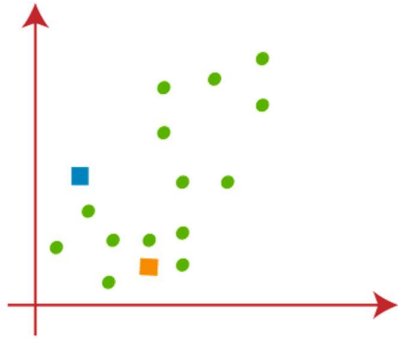
Let's understand the above steps by considering the visual plots:

Suppose we have two variables $M1$ and $M2$. The x-y axis scatter plot of these two variables is given below:

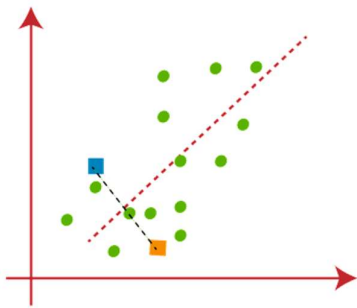


- Let's take number k of clusters, i.e., $K=2$, to identify the dataset and to put them into different clusters. It means here we will try to group these datasets into two different clusters.
- We need to choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point. So, here we are selecting the below two points as k points, which are not

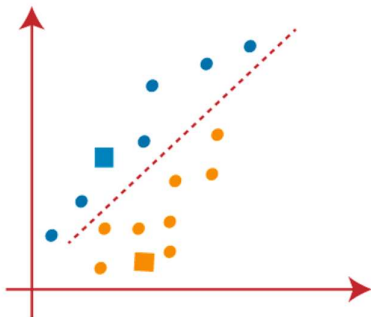
the part of our dataset. Consider the below image:



- Now we will assign each data point of the scatter plot to its closest K-point or centroid. We will compute it by applying some mathematics that we have studied to calculate the distance between two points. So, we will draw a median between both the centroids. Consider the below image:

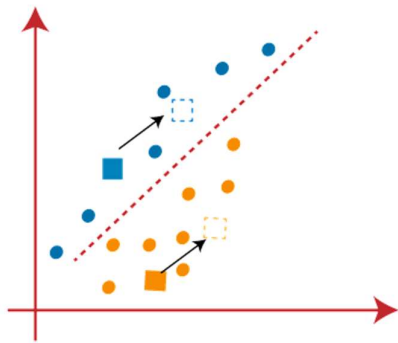


From the above image, it is clear that points left side of the line is near to the K1 or blue centroid, and points to the right of the line are close to the yellow centroid. Let's colour them as blue and yellow for clear visualization.

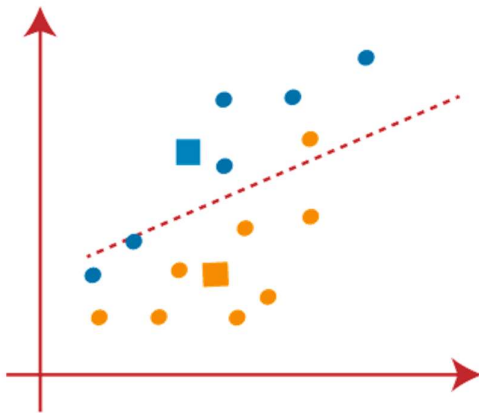


- As we need to find the closest cluster, so we will repeat the process by choosing a **new centroid**. To choose the new centroids, we will compute

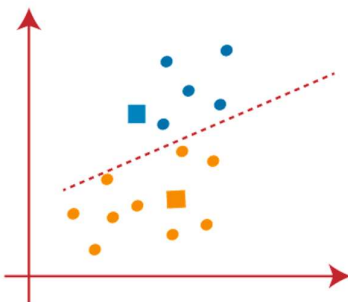
the centre of gravity of these centroids, and will find new centroids as below:



Next, we will reassign each datapoint to the new centroid. For this, we will repeat the same process of finding a median line. The median will be like below image:

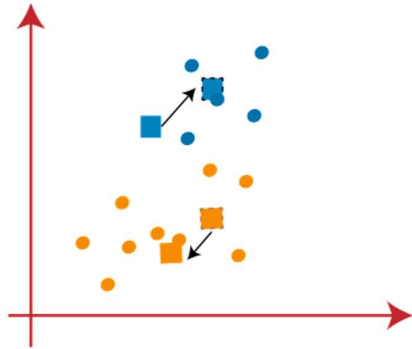


From the above image, we can see, one yellow point is on the left side of the line, and two blue points are right to the line. So, these three points will be assigned to new centroids.

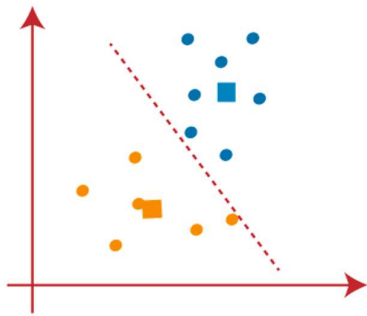


As reassignment has taken place, so we will again go to the step-4, which is finding new centroids or K-points.

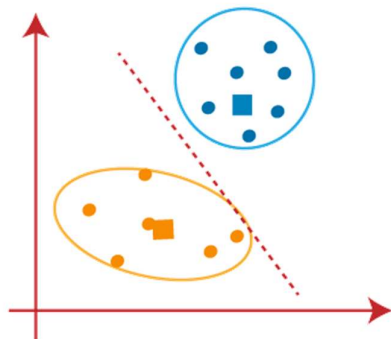
- We will repeat the process by finding the centre of gravity of centroids, so the new centroids will be as shown in the below image:



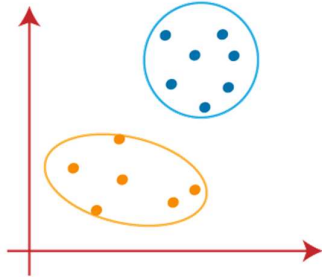
- As we got the new centroids so again will draw the median line and reassign the data points. So, the image will be:



- We can see in the above image; there are no dissimilar data points on either side of the line, which means our model is formed. Consider the below image:



As our model is ready, so we can now remove the assumed centroids, and the two final clusters will be as shown in the below image:



How to choose the value of "K number of clusters" in K-means Clustering?

The performance of the K-means clustering algorithm depends upon highly efficient clusters that it forms. But choosing the optimal number of clusters is a big task. There are some different ways to find the optimal number of clusters, but here we are discussing the most appropriate method to find the number of clusters or value of K. The method is given below:

Elbow Method

The Elbow method is one of the most popular ways to find the optimal number of clusters. This method uses the concept of TWSS value. **TWSS** stands for **Total Within Sum of Squares**, which defines the total variations within a cluster. The formula to calculate the value of TWSS (for 3 clusters) is given below:

$$TWSS = \sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2 + \sum_{P_i \text{ in Cluster2}} \text{distance}(P_i C_2)^2 + \sum_{P_i \text{ in Cluster3}} \text{distance}(P_i C_3)^2$$

In the above formula of TWSS,

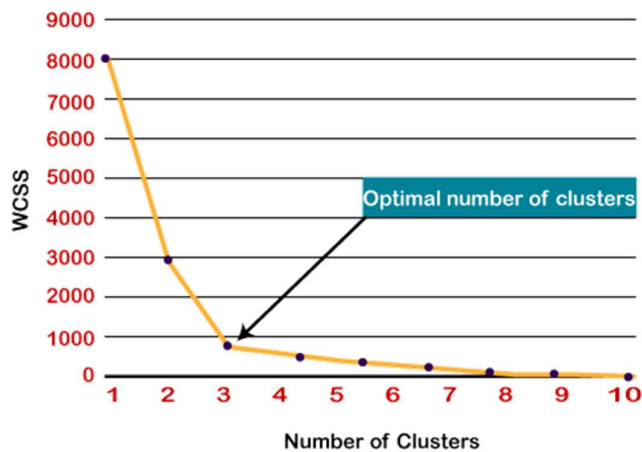
$\sum_{P_i \text{ in Cluster1}} \text{distance}(P_i C_1)^2$: It is the sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the other two terms.

To measure the distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance.

To find the optimal value of clusters, the elbow method follows the below steps:

- It executes the K-means clustering on a given dataset for different K values (ranges from 1-10).
- For each value of K, calculates the TWSS value.
- Plots a curve between calculated TWSS values and the number of clusters K.
- The sharp point of bend or a point of the plot looks like an arm, then that point is considered as the best value of K.

Since the graph shows the sharp bend, which looks like an elbow, hence it is known as the elbow method. The graph for the elbow method looks like the below image:



Silhouette Analysis

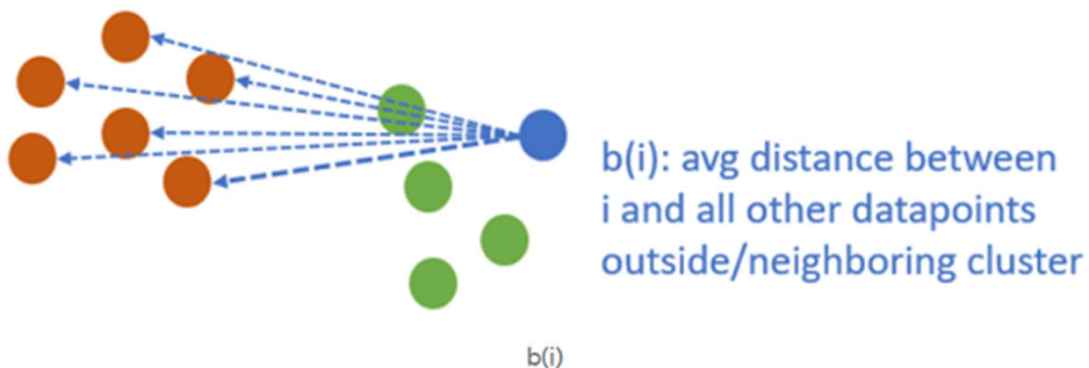
The silhouette coefficient or silhouette score k means is a measure of how similar a data point is within-cluster (cohesion) compared to other clusters (separation). The Silhouette score can be easily calculated in Python using the metrics module of the scikit-learn/sklearn library.

- Select a range of values of k (say 1 to 10).
- Plot Silhouette coefficient for each value of K .

The equation for calculating the silhouette coefficient for a particular data point:

- $S(i)$ is the silhouette coefficient of the data point i .
- $a(i)$ is the average distance between i and all the other data points in the cluster to which i belongs.
- $b(i)$ is the average distance from i to all clusters to which i does not belong.

Source: medium



Source: medium

We will then calculate the average_silhouette for every k .

$$\text{AverageSilhouette} = \text{mean}\{S(i)\}$$

Then plot the graph between average_silhouette and K.

Points to Remember While Calculating Silhouette Coefficient:

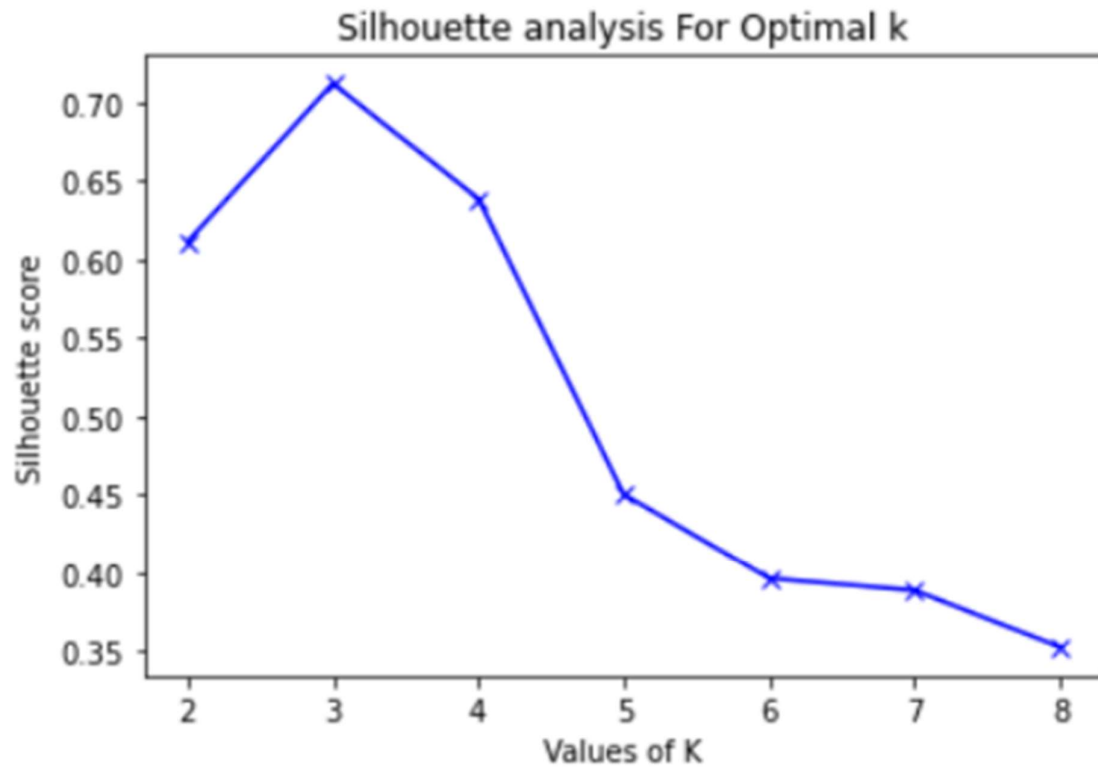
- The value of the silhouette coefficient is between [-1, 1].
- A score of 1 denotes the best, meaning that the data point i is very compact within the cluster to which it belongs and far away from the other clusters.
- The worst value is -1. Values near 0 denote overlapping clusters.

Let us see the python code with the help of an example.

```
range_n_clusters = [2, 3, 4, 5, 6, 7, 8]
silhouette_avg = []
for num_clusters in range_n_clusters:

    # initialise kmeans
    kmeans = KMeans(n_clusters=num_clusters)
    kmeans.fit(data_frame)
    cluster_labels = kmeans.labels_

    # silhouette score
    silhouette_avg.append(silhouette_score(data_frame,
    cluster_labels))plt.plot(range_n_clusters,silhouette_avg,'bx-')
plt.xlabel('Values of K')
plt.ylabel('Silhouette score')
plt.title('Silhouette analysis For Optimal k')
plt.show()
```



Line plot between K and Silhouette score

We see that the silhouette score is maximized at $k = 3$. So, we will take 3 clusters.

NOTE: The silhouette Method is used in combination with the Elbow Method for a more confident decision.

K-Medoids Algorithm

K-Medoids is an unsupervised clustering algorithm in which data points called “**medoids**” act as the cluster's center. A medoid is a point in the cluster whose sum of distances(also called dissimilarity) to all the objects in the cluster is minimal. The distance can be the Euclidean distance, Manhattan distance, or any other suitable distance function.

Therefore, the K -medoids algorithm divides the data into K clusters by selecting K medoids from our data sample.

Working of the Algorithm:

The steps taken by the **K-medoids algorithm** for clustering can be explained as follows:-

1. Randomly select k points from the data(k is the number of clusters to be formed). These k points would act as our initial medoids.
2. The distances between the medoid points and the non-medoid points are calculated, and each point is assigned to the cluster of its nearest medoid.
3. Calculate the cost as the total sum of the distances(also called dissimilarities) of the data points from the assigned medoid.
4. Swap one medoid point with a non-medoid point(from the same cluster as the medoid point) and recalculate the cost.
5. If the calculated cost with the new medoid point is more than the previous cost, we undo the swap, and the algorithm converges else; we repeat step 4

Finally, we will have k medoid points with their clusters.

Let us understand the working of the algorithm with the help of an example. For the below example, I will be using Manhattan Distance as the distance metric for calculating the distance between the points.

Manhattan Distance between two points (x1,y1) and (x2,y2) is given as

$$\mathbf{Mdist} = |x2 - x1| + |y2 - y1|$$

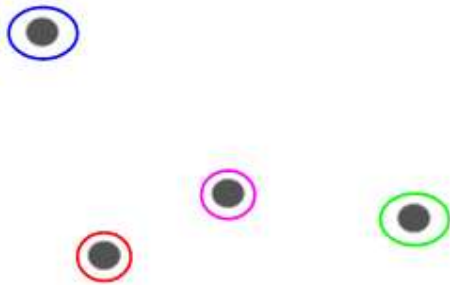
Hierarchical Clustering

What is Hierarchical Clustering?

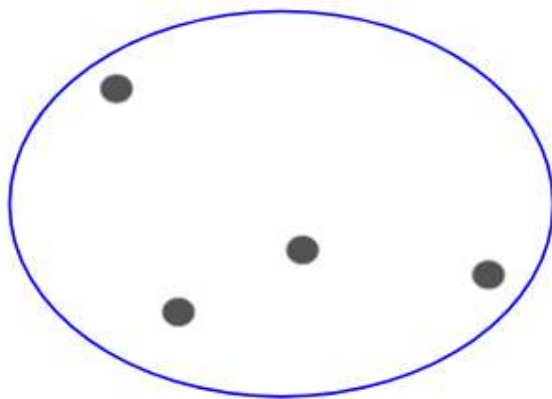
Let us say we have the below points and we want to cluster them into groups:



We can assign each of these points to a separate cluster:



Now, based on the similarity of these clusters, we can combine the most similar clusters together and repeat this process until only a single cluster is left:



We are essentially building a hierarchy of clusters. That is why this algorithm is called hierarchical clustering. I will discuss how to decide the number of clusters in a later section. For now, let us look at the different types of hierarchical clustering.

Types of Hierarchical Clustering

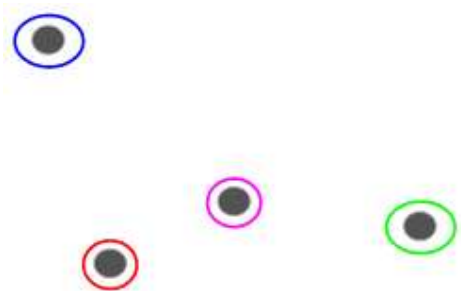
There are mainly two types of hierarchical clustering:

1. Agglomerative hierarchical clustering
2. Divisive Hierarchical clustering

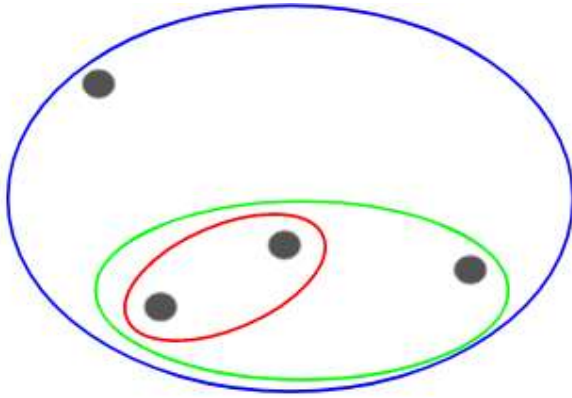
Let's understand each type in detail.

Agglomerative Hierarchical Clustering

We assign each point to an individual cluster in this technique. Suppose there are 4 data points. We will assign each of these points to a cluster and hence will have 4 clusters in the beginning:



Then, at each iteration, we merge the closest pair of clusters and repeat this step until only a single cluster is left:

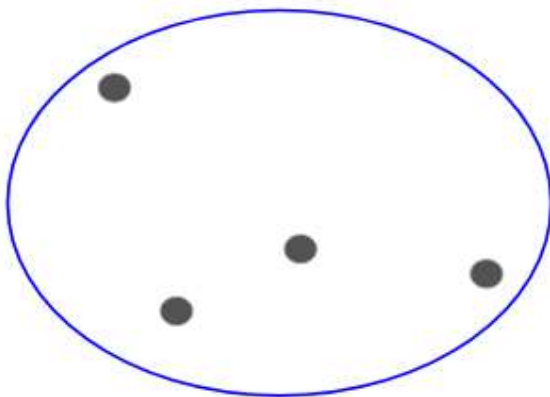


We are merging (or adding) the clusters at each step, right? Hence, this type of clustering is also known as **additive hierarchical clustering**.

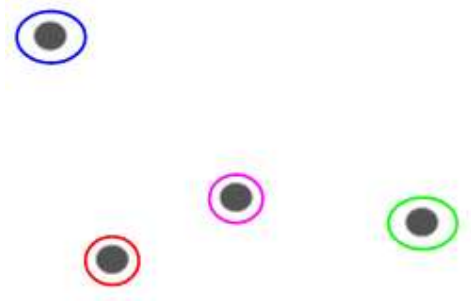
Divisive Hierarchical Clustering

Divisive hierarchical clustering works in the opposite way. Instead of starting with n clusters (in case of n observations), we start with a single cluster and assign all the points to that cluster.

So, it doesn't matter if we have 10 or 1000 data points. All these points will belong to the same cluster at the beginning:



Now, at each iteration, we split the farthest point in the cluster and repeat this process until each cluster only contains a single point:



We are splitting (or dividing) the clusters at each step, hence the name divisive hierarchical clustering.

Agglomerative Clustering is widely used in the industry and that will be the focus in this article. Divisive hierarchical clustering will be a piece of cake once we have a handle on the agglomerative type.

Steps to Perform Hierarchical Clustering

We merge the most similar points or clusters in hierarchical clustering – we know this. Now the question is – how do we decide which points are similar and which are not? It's one of the most important questions in clustering!

Here's one way to calculate similarity – Take the distance between the centroids of these clusters. The points having the least distance are referred to as similar points and we can merge them. We can refer to this as a **distance-based algorithm** as well (since we are calculating the distances between the clusters).

In hierarchical clustering, we have a concept called a **proximity matrix**. This stores the distances between each point. Let's take an example to understand this matrix as well as the steps to perform hierarchical clustering.

Setting up the Example



Suppose a teacher wants to divide her students into different groups. She has the marks scored by each student in an assignment and based on these marks, she wants to segment them into groups. There's no fixed target here as to how many groups to have. Since the teacher does not know what type of students should be assigned to which group, it cannot be solved as a supervised learning problem. So, we will try to apply hierarchical clustering here and segment the students into different groups.

Let's take a sample of 5 students:

Student_ID	Marks
1	10
2	7
3	28
4	20
5	35

Creating a Proximity Matrix

First, we will create a proximity matrix which will tell us the distance between each of these points. Since we are calculating the distance of each point from each of the other points, we will get a square matrix of shape $n \times n$ (where n is the number of observations).

Let's make the 5×5 proximity matrix for our example:

ID	1	2	3	4	5
1	0	3	18	10	25
2	3	0	21	13	28
3	18	21	0	8	7
4	10	13	8	0	15
5	25	28	7	15	0

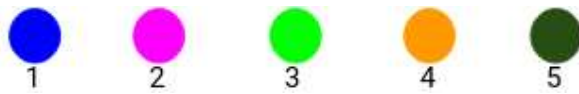
The diagonal elements of this matrix will always be 0 as the distance of a point with itself is always 0. We will use the Euclidean distance formula to calculate the rest of the distances. So, let's say we want to calculate the distance between point 1 and 2:

$$\sqrt{(10-7)^2} = \sqrt{9} = 3$$

Similarly, we can calculate all the distances and fill the proximity matrix.

Steps to Perform Hierarchical Clustering

Step 1: First, we assign all the points to an individual cluster:

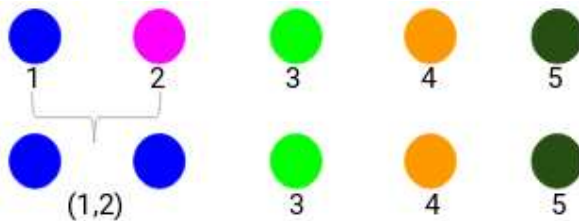


Different colors here represent different clusters. You can see that we have 5 different clusters for the 5 points in our data.

Step 2: Next, we will look at the smallest distance in the proximity matrix and merge the points with the smallest distance. We then update the proximity matrix:

ID	1	2	3	4	5
1	0	3	18	10	25
2	3	0	21	13	28
3	18	21	0	8	7
4	10	13	8	0	15
5	25	28	7	15	0

Here, the smallest distance is 3 and hence we will merge point 1 and 2:



Let's look at the updated clusters and accordingly update the proximity matrix:

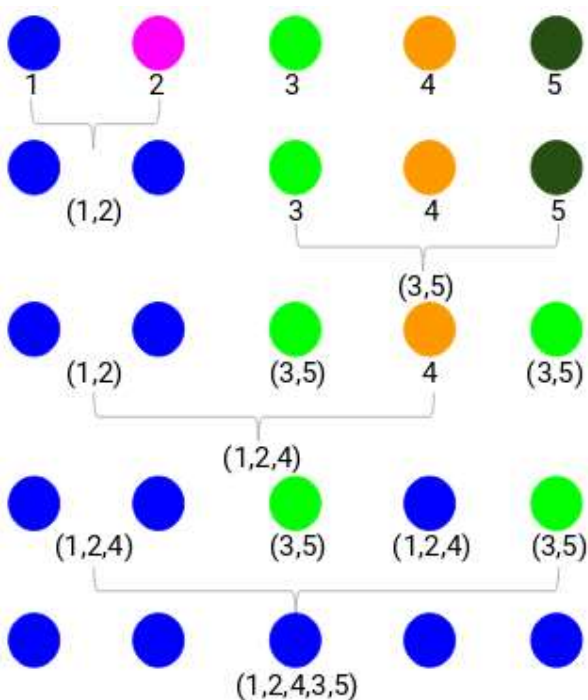
Student_ID	Marks
(1,2)	10
3	28
4	20
5	35

Here, we have taken the maximum of the two marks (7, 10) to replace the marks for this cluster. Instead of the maximum, we can also take the minimum value or the average values as well. Now, we will again calculate the proximity matrix for these clusters:

ID	(1,2)	3	4	5
(1,2)	0	18	10	25
3	18	0	8	7
4	10	8	0	15
5	25	7	15	0

Step 3: We will repeat step 2 until only a single cluster is left.

So, we will first look at the minimum distance in the proximity matrix and then merge the closest pair of clusters. We will get the merged clusters as shown below after repeating these steps:



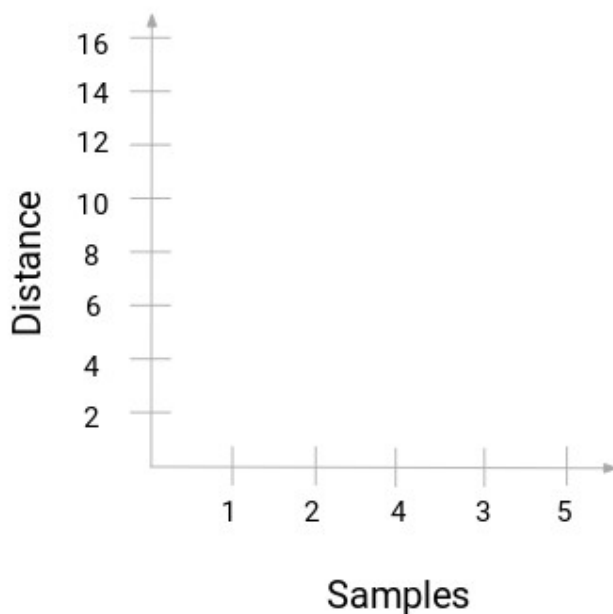
We started with 5 clusters and finally have a single cluster. **This is how agglomerative hierarchical clustering works.** But the burning question still remains – how do we decide the number of clusters? Let's understand that in the next section.

How should we Choose the Number of Clusters in Hierarchical Clustering?

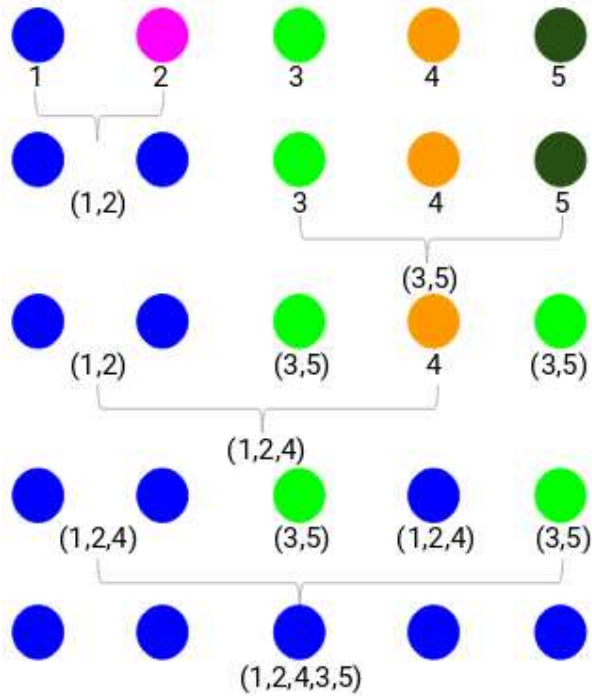
Ready to finally answer this question that's been hanging around since we started learning? To get the number of clusters for hierarchical clustering, we make use of an awesome concept called a **Dendrogram**.

A dendrogram is a tree-like diagram that records the sequences of merges or splits.

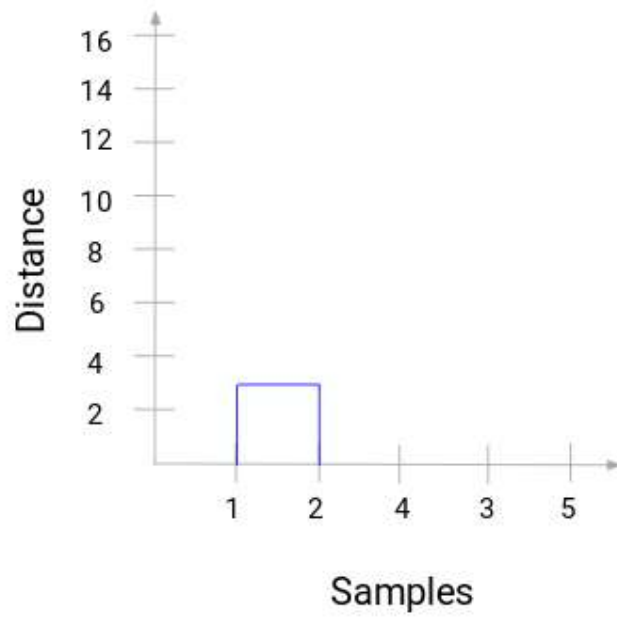
Let's get back to our teacher-student example. Whenever we merge two clusters, a dendrogram will record the distance between these clusters and represent it in graph form. Let's see how a dendrogram looks like:



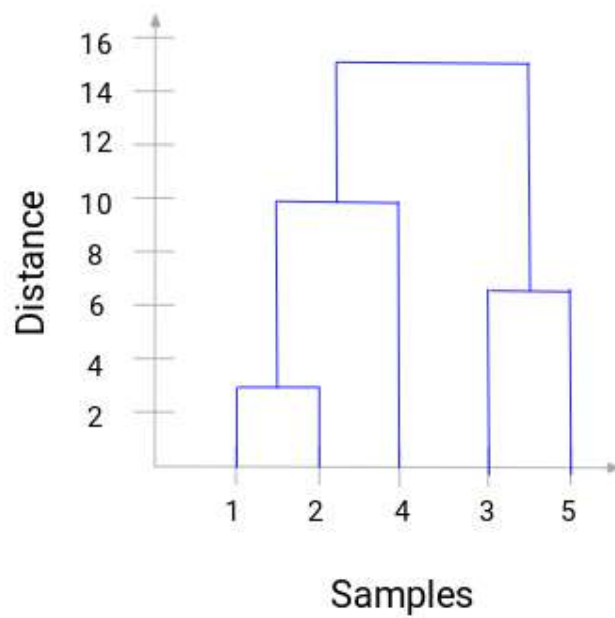
We have the samples of the dataset on the x-axis and the distance on the y-axis. **Whenever two clusters are merged, we will join them in this dendrogram and the height of the join will be the distance between these points.** Let's build the dendrogram for our example:



Take a moment to process the above image. We started by merging sample 1 and 2 and the distance between these two samples was 3 (refer to the first proximity matrix in the previous section). Let's plot this in the dendrogram:

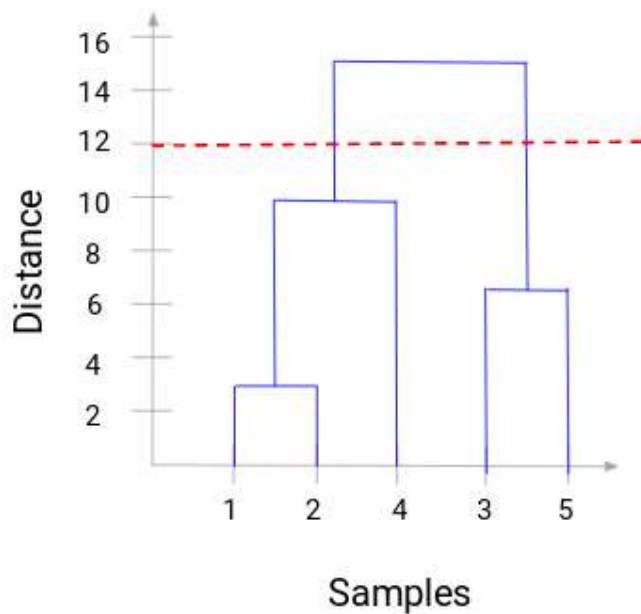


Here, we can see that we have merged sample 1 and 2. The vertical line represents the distance between these samples. Similarly, we plot all the steps where we merged the clusters and finally, we get a dendrogram like this:



We can clearly visualize the steps of hierarchical clustering. **More the distance of the vertical lines in the dendrogram, more the distance between those clusters.**

Now, we can set a threshold distance and draw a horizontal line (*Generally, we try to set the threshold in such a way that it cuts the tallest vertical line*). Let's set this threshold as 12 and draw a horizontal line:



The number of clusters will be the number of vertical lines which are being intersected by the line drawn using the threshold. In the above example, since the red line intersects 2 vertical lines, we will have 2 clusters. One cluster will have a sample (1,2,4) and the other will have a sample (3,5). Pretty straightforward, right?

This is how we can decide the number of clusters using a dendrogram in Hierarchical Clustering.

Principal Component Analysis (PCA)

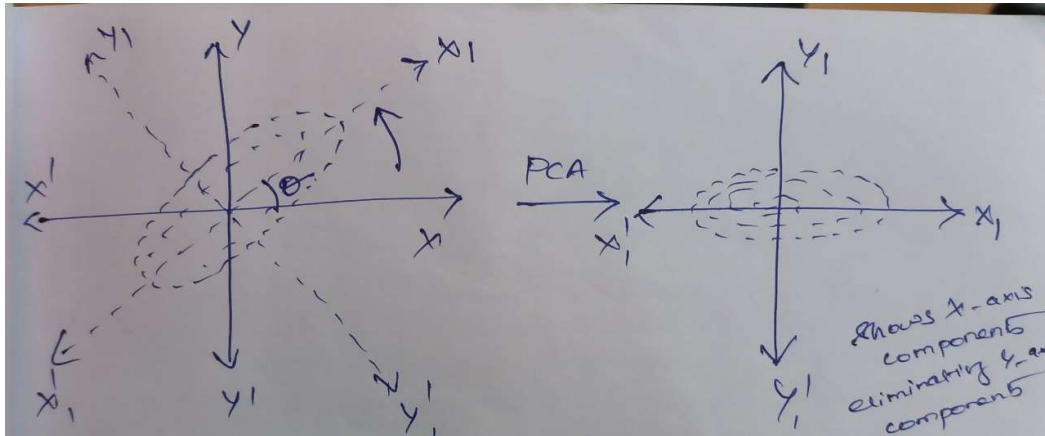
PCA, or Principal Component Analysis, is a term that is well-known to everyone. Notably employed for Curse of Dimensionality issues. In addition to this fundamental issue, there are other significant issues that we tackle in the PCA article. So, let's start with fundamental knowledge. In this article, I've also added my handwritten manual technique for PCA, layman comprehension, some key theory, and a Python approach.

Principal Component Analysis (PCA)

1. PCA can be abbreviated as Principal Component Analysis
2. PCA comes under the Unsupervised Machine Learning category
3. Reducing the number of variables in a data collection while retaining as much information as feasible is the main goal of PCA. PCA can be mainly used for Dimensionality Reduction and also for important feature selection.
4. Correlated features to independent features

What is Principal Component Analysis?

Technically, PCA provides a complete explanation of the composition of variance and covariance using multiple linear combinations of the core variables. Row scattering may be analyzed using PCA, which also identifies the distribution-related properties.



Why do we need PCA?

When a computer is trained on a big, well-organized dataset, machine learning often excels. One of the techniques used to handle the curse of dimensionality in machine learning is principal component analysis (PCA). Typically, having a sufficient amount of data enables us to create a more accurate prediction model since we have more data to use to train the computer. But working with a huge data collection has its own drawbacks. The curse of dimensionality is the ultimate trap.

The title of an unreleased Harry Potter novel does not refer to what happens when your data has too many characteristics and perhaps not enough data points; rather, it refers to the curse of dimensionality. One can use dimensionality reduction to escape the dimensionality curse. Having 50 variables may be cut down to 40, 20, or even 10. The strongest effects of dimensionality reduction are found here.

Overfitting issues will arise while working with high-dimensional data, and dimensionality reduction will be used to address them. increasing interpretability and minimizing information loss. aids in locating important characteristics. aids in the discovery of a linear combination of varied sequences.

When to use PCA?

1. Whenever we need to know our features are independent of each other
2. Whenever we need fewer features from higher features

Dimensionality Reduction Work in Real-Time Application

Assume there are 50 questions in all in the survey. The following three are among them: Please give the following a rating between 1 and 5:

1. I feel comfortable around people
2. I easily make friends
3. I like going out

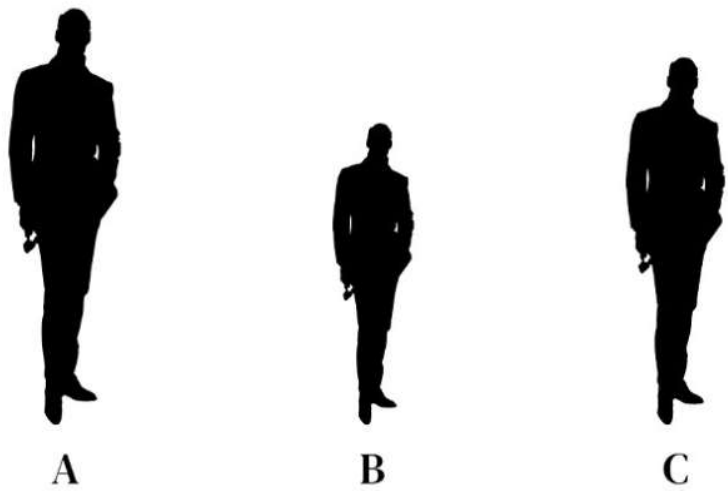
These queries could appear different now. There is a catch, though. They aren't, generally speaking. They all gauge how extroverted you are. Therefore, combining them makes it logical, right? That's where linear algebra and dimensionality reduction methods come in! We want to lessen the complexity of the problem by minimizing the number of variables since we have much too many variables that aren't all that different. That is the main idea behind [dimensionality reduction](#). And it just so happens that PCA is one of the most straightforward and popular techniques in this field. As a general guideline, maintain at least 70–80 percent of the explained variation.

Intuition behind PCA

Let's assume we are playing a mind game here like,

Person	Height
A	145
B	160
C	185

from the above table, we need to find the tallest person.



I can by seeing person A is the tallest. Now change the scenario

Person	Height
D	172
E	173
F	171

Can you guess who's who? It's tough when they are very similar in height.

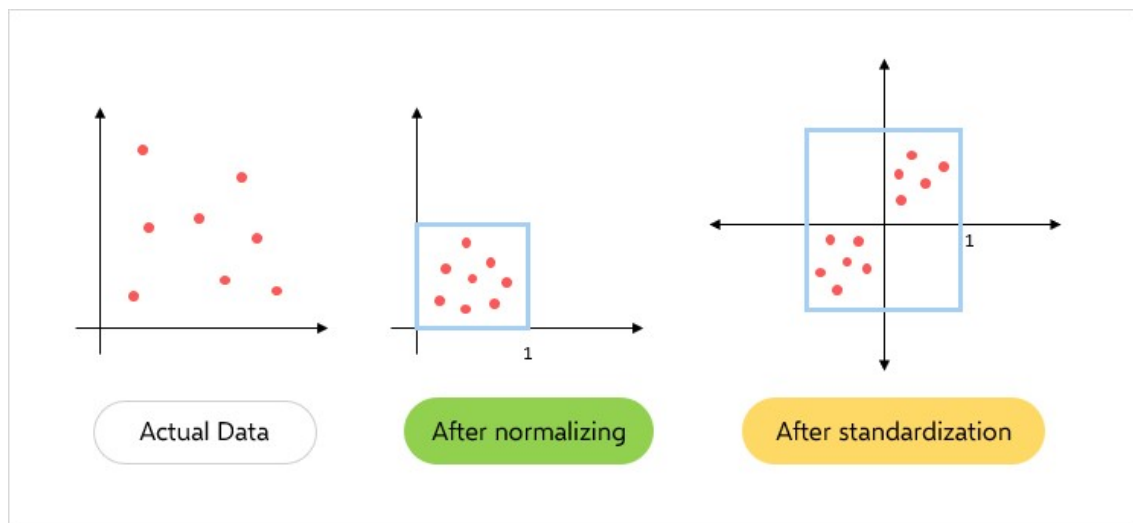


Because of how much their heights vary, we previously had no issue telling a 185cm person from a 160cm and a 145cm person. Similar to this, our data contains more information when its variance is bigger. This explains why the terms PCA and maximum variance are frequently used together.

Basic Terminologies of PCA

Before getting into PCA, we need to understand some basic terminologies,

- **Variance** – for calculating the variation of data distributed across dimensionality of graph
- **Covariance** – calculating dependencies and relationship between features
- **Standardizing data** – Scaling our dataset within a specific range for unbiased output



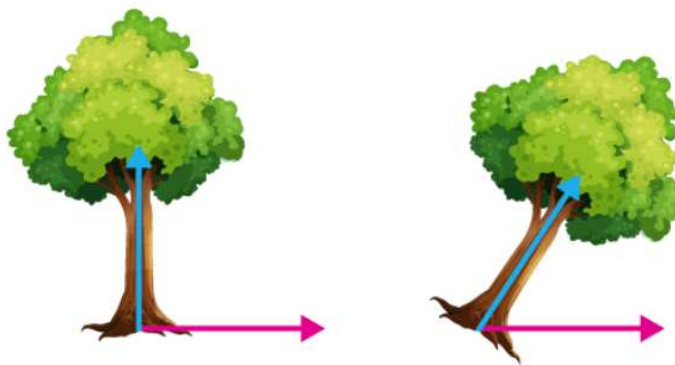
- **Covariance matrix** – Used for calculating interdependencies between the features or variables and also helps in reduce it to improve the performance

$$Cov(x, y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{n}$$

Diagram illustrating the components of the covariance formula:

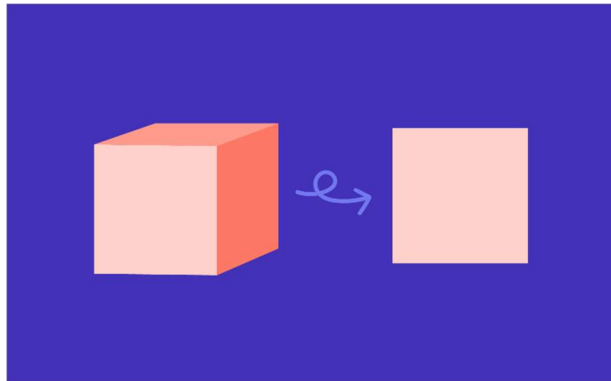
- x_i : data value of X
- $\bar{x}$: mean value of X
- y_i : data value of Y
- $\bar{y}$: mean value of Y
- n : Number of data values

- **EigenValues and EigenVectors** – Eigenvectors' purpose is to find out the largest variance that exists in the dataset to calculate Principal Component. Eigenvalue means the magnitude of the Eigenvector. Eigenvalue indicates variance in a particular direction and whereas eigenvector is expanding or contracting X-Y (2D) graph without altering the direction.



In this shear mapping, the blue arrow changes direction whereas the pink arrow does not. The pink arrow in this instance is an eigenvector because of its constant orientation. The length of this arrow is also unaltered, and its eigenvalue is 1. Technically, PC is a straight line that captures the maximum variance (information) of the data. PC shows direction and magnitude. PC are perpendicular to each other.

- **Dimensionality Reduction** – Transpose of original data and multiply it by transposing of the derived feature vector. Reducing the features without losing information.



How does PCA work?

The steps involved for PCA are as follows-

1. Original Data
2. Normalize the original data (mean =0, variance =1)
3. Calculating covariance matrix
4. Calculating Eigen values, Eigen vectors, and normalized Eigenvectors
5. Calculating Principal Component (PC)
6. Plot the graph for orthogonality between PCs

I have solved through manually, and the importance of hand-written notes is getting the crux behind the coding concepts,

Consider dataset,

Feature	sample1	sample2	sample3	sample4
a	4	8	13	7
b	11	4	5	14

Step1:

No. of features, $n = 2$ (a, b)

No. of samples, $N = 4$ (sample1, sample2, sample3, sample4)

Step2:

calculating mean,

$$\bar{a} = \frac{4+8+13+7}{4} = 8$$

$$\bar{b} = \frac{11+4+5+14}{4} = 8.5$$

Step3:

calculating covariance matrix, between features,

In the given dataset, ordered features are as,

(a, a), (a, b), (b, a), (b, b)

$$\begin{aligned} \text{cov}(a, a) &= \frac{1}{N-1} \sum_{k=1}^N (a_i - \bar{a})(a_i - \bar{a}) \\ &= \frac{1}{N-1} \sum_{k=1}^N (a_i - \bar{a})^2 \rightarrow \text{for same feature} \\ &= \frac{1}{4-1} [(4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2] \\ &= \frac{4^2 + 0 + 5^2 + 1^2}{3} = \frac{16+0+25+1}{3} \\ &= \frac{42}{3} = 14. \end{aligned}$$

$$\begin{aligned} \text{cov}(a, b) &= \frac{1}{N-1} \sum_{k=1}^N (a_i - \bar{a})(b_i - \bar{b}) \\ &= \frac{1}{4-1} [(4-8)(11-8.5) + (8-8)(4-8.5) + \\ &\quad (13-8)(5-8.5) + (7-8)(14-8.5)] \\ &= \frac{1}{3} [(-4)(2.5) + (0) + (5)(-3.5) + (-1)(5.5)] \\ &= \frac{1}{3} [-10 - 17.5 - 5.5] = \frac{-33}{3} = -11 \end{aligned}$$

We are calculating means and then calculating the covariance matrix between features.

$$\text{cov}(b, a) = \frac{1}{N-1} \sum_{k=1}^N (b_i - \bar{b})(a_i - \bar{a})$$

$$= \text{cov}(a, b)$$

$$= -11$$

$$\text{cov}(b, b) = \frac{1}{N-1} \sum_{k=1}^N (b_i - \bar{b})(b_i - \bar{b})$$

$$= \frac{1}{N-1} \sum_{k=1}^N (b_i - \bar{b})^2$$

$$= \frac{1}{4-1} [(11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (14-8.5)^2]$$

$$= \frac{1}{3} [(2.5)^2 + (-4.5)^2 + (-3.5)^2 + (5.5)^2]$$

$$= \frac{1}{3} [6.25 + 20.25 + 12.25 + 30.25]$$

$$= \frac{69}{3} = \underline{\underline{23}}$$

Hence covariance matrix can be

$$S = \begin{bmatrix} \text{cov}(a, a) & \text{cov}(a, b) \\ \text{cov}(b, a) & \text{cov}(b, b) \end{bmatrix} = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Step 4:-

calculate Eigen value, Eigen vectors, Normalized Eigen vectors.

In order calculate Eigen value,

$$\det(S - \lambda I) = 0$$

$$I (\text{Identity matrix}) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\lambda I = \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}$$

$$\det \left(\begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} \right) = 0$$

$$\det \begin{pmatrix} 14-\lambda & -11 \\ -11 & 23-\lambda \end{pmatrix} = 0.$$

$$(14-\lambda)(23-\lambda) - (-11 \times -11) = 0$$

$$322 - 14\lambda - 23\lambda + \lambda^2 - 121 = 0$$

After finding covariance matrix, we are going to calculate the eigenvalue, eigenvector, and normalized eigenvector

$$201 - 37\lambda + \lambda^2 = 0.$$

After rearranging,

$$\lambda^2 - 37\lambda + 201 = 0.$$

' λ ' can be calculated by quadratic eqn,

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad \begin{array}{l} a=1 \\ b=-37 \\ c=201 \end{array}$$

$$= \frac{-(-37) \pm \sqrt{(-37)^2 - 4(1)(201)}}{2(1)}$$

$$= \frac{37 \pm \sqrt{1369 - 804}}{2} = \frac{37 \pm \sqrt{565}}{2}$$

$$= \frac{37 \pm 23.76}{2} \Rightarrow \frac{37+23.76}{2}, \frac{37-23.76}{2}$$

$$= \frac{60.76}{2}, \frac{13.24}{2}$$

Eigen values. $\boxed{\lambda_1 = 30.38, \lambda_2 = 6.62}$

So, while arranging in descending order,

$$\lambda_1 > \lambda_2 > \dots$$

Hence, $\lambda_1 = 30.38$

$$\lambda_2 = 6.62$$

Now we are going to find out Eigenvectors for

Eigen value, $\lambda = 30.38$.

$$(S - \lambda_1 I) U_1 = 0$$

Covariance matrix 30.38 Identity matrix

Eigenvectors of λ_1

$$\left(\begin{pmatrix} 14 & -11 \\ -11 & 23 \end{pmatrix} - 30.38 \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right) U_1 = 0$$

$$\text{Assume } U_1 = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

Hence,

$$\begin{pmatrix} 14 & -11 \\ -11 & 23 \end{pmatrix} - \begin{pmatrix} 30.38 & 0 \\ 0 & 30.38 \end{pmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Steps involved in eigenvalues and vectors, in the manual approach

$$\begin{pmatrix} 14 - 30.38 & -11 \\ -11 & 23 - 30.38 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$\begin{pmatrix} -16.38 & -11 \\ -11 & -7.38 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$-16.38 u_1 - 11 u_2 = 0 \rightarrow (1)$$

$$-11 u_1 - 7.38 u_2 = 0 \rightarrow (2)$$

So, from this if we need to calculate u_1 & u_2

$$(1) \times 7.38 \Rightarrow 120.88 u_1 - 81.12 u_2 = 0$$

$$(2) \times -11 \Rightarrow +121 u_1 + 81.18 u_2 = 0$$

$$0.12 u_1 = 0$$

$$u_1 = 0$$

then, apply u_1 in (1), then

$$-16.38 \times 0 - 11 u_2 = 0$$

$$u_2 = 0$$

this can't be possible, hence

$$\begin{bmatrix} (14 - \lambda_1) & (-11) \\ (-11) & (23 - \lambda_1) \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$(14 - \lambda_1) u_1 - 11 u_2 = 0 \rightarrow (a)$$

$$-11 u_1 + (23 - \lambda_1) u_2 = 0 \rightarrow (b)$$

from (a),

$$(14 - \lambda_1) u_1 - 11 u_2 = 0$$

$$(14 - \lambda_1) u_1 = 11 u_2$$

$$\frac{u_1}{11} = \frac{u_2}{14 - \lambda_1} = A \text{ (Assigning)}$$

Assume $A=1$,

$$\frac{u_1}{11} = \frac{u_2}{14 - \lambda_1} = A = 1$$

$$\text{Hence, } \frac{u_1}{11} = 1 \Rightarrow u_1 = 11$$

$$\begin{aligned} \frac{u_2}{14 - \lambda_1} &= 1 \Rightarrow u_2 = 14 - \lambda_1 \\ &= 14 - 30.38 \\ &= -16.38 \end{aligned}$$

$$\text{Hence Eigen vector for } \lambda_1 \Rightarrow \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 11 \\ -16.38 \end{bmatrix}$$

From this, we are going to calculate PCs

Then if we want to normalize the eigen vectors,

$$n_1 = \begin{bmatrix} 11 / \sqrt{11^2 + 16.38^2} \\ -16.38 / \sqrt{11^2 + 16.38^2} \end{bmatrix} \quad / \text{dividing by the length.}$$

$$= \begin{bmatrix} \frac{11}{19.73} \\ -16.38 / 19.73 \end{bmatrix} = \begin{bmatrix} 0.5575 \\ -0.8302 \end{bmatrix}$$

Now, calculate eigen vectors for $\lambda_2 = 6.62$

$$(S - \lambda_2 I) v_2 = 0$$

$$\begin{bmatrix} (14 - \lambda_2) & -11 \\ -11 & (23 - \lambda_2) \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$(14 - \lambda_2) v_1 - 11 v_2 = 0 \rightarrow (c)$$

$$-11 v_1 - (23 - \lambda_2) v_2 = 0 \rightarrow (d)$$

From (c),

$$(14 - \lambda_2) v_1 - 11 v_2 = 0$$

$$(14 - \lambda_2) v_1 = 11 v_2$$

$$\frac{v_1}{11} = \frac{v_2}{14 - \lambda_2} = B \text{ (Assume)}$$

Assume $B = 1$,

$$\frac{v_1}{11} = \frac{v_2}{14 - \lambda_2} = B = 1$$

$$\text{Hence, } \frac{v_1}{11} = 1 \Rightarrow v_1 = 11$$

$$\frac{v_2}{14 - \lambda_2} = 1 \Rightarrow v_2 = 14 - \lambda_2$$

$$= 14 - 6.62$$

$$= 7.38$$

$$\text{Hence, Eigen Vectors for } \lambda_2 = \begin{bmatrix} 11 \\ 7.38 \end{bmatrix}$$

If we want to normalize eigen vectors,

$$n_2 = \begin{bmatrix} 11 / \sqrt{11^2 + 7.38^2} \\ 7.38 / \sqrt{11^2 + 7.38^2} \end{bmatrix} = \begin{bmatrix} 11 / 13.24 \\ 7.38 / 13.24 \end{bmatrix}$$

$$= \begin{bmatrix} 0.8308 \\ 0.5574 \end{bmatrix}$$

We are going to calculate the normalized eigenvector

Step 5: New dataset,

Feature	sample1	sample2	sample3	sample4
a	4	8	13	7
b	11	4	5	14

1st PC	P ₁₁	P ₁₂	P ₁₃	P ₁₄
	sample1	sample2	sample3	sample4

$$P_{11} = n_1^T \begin{bmatrix} 4-8 \\ 11-8.5 \end{bmatrix}$$

$$= \begin{bmatrix} 0.5575 & -0.8302 \end{bmatrix} \begin{bmatrix} -4 \\ 2.5 \end{bmatrix}$$

$$= (-2.23 - 2.0755)$$

$$= -4.305$$

$$P_{12} = n_1^T \begin{bmatrix} 8-8 \\ 4-8.5 \end{bmatrix} \Rightarrow (0.5575 - 0.8302) \begin{pmatrix} 0 \\ -4.5 \end{pmatrix}$$

$$= 0 + 3.7359 = 3.7359$$

$$P_{13} = n_1^T \begin{bmatrix} 13-8 \\ 5-8.5 \end{bmatrix} \Rightarrow (0.5575 - 0.8302) \begin{pmatrix} 5 \\ -3.5 \end{pmatrix}$$

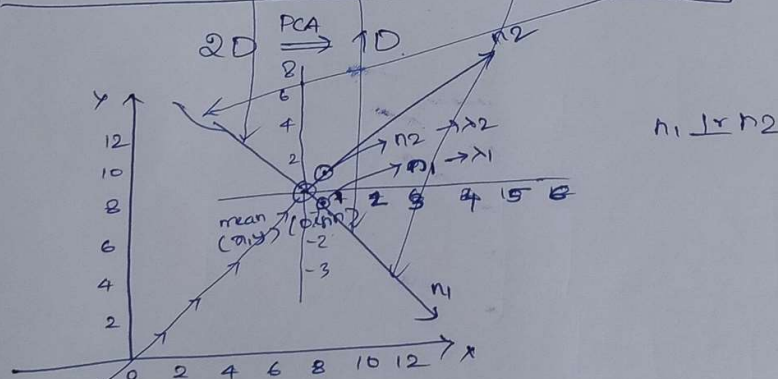
$$= 2.7875 + 2.9057 = 5.692$$

$$P_{14} = n_1^T \begin{bmatrix} 7-8 \\ 14-8.5 \end{bmatrix} = (0.5575 - 0.8302) \begin{pmatrix} -1 \\ 5.5 \end{pmatrix}$$

$$= -0.5575 - 4.5661$$

$$= -5.123$$

PC1	-4.305	3.7359	5.692	-5.123
-----	--------	--------	-------	--------



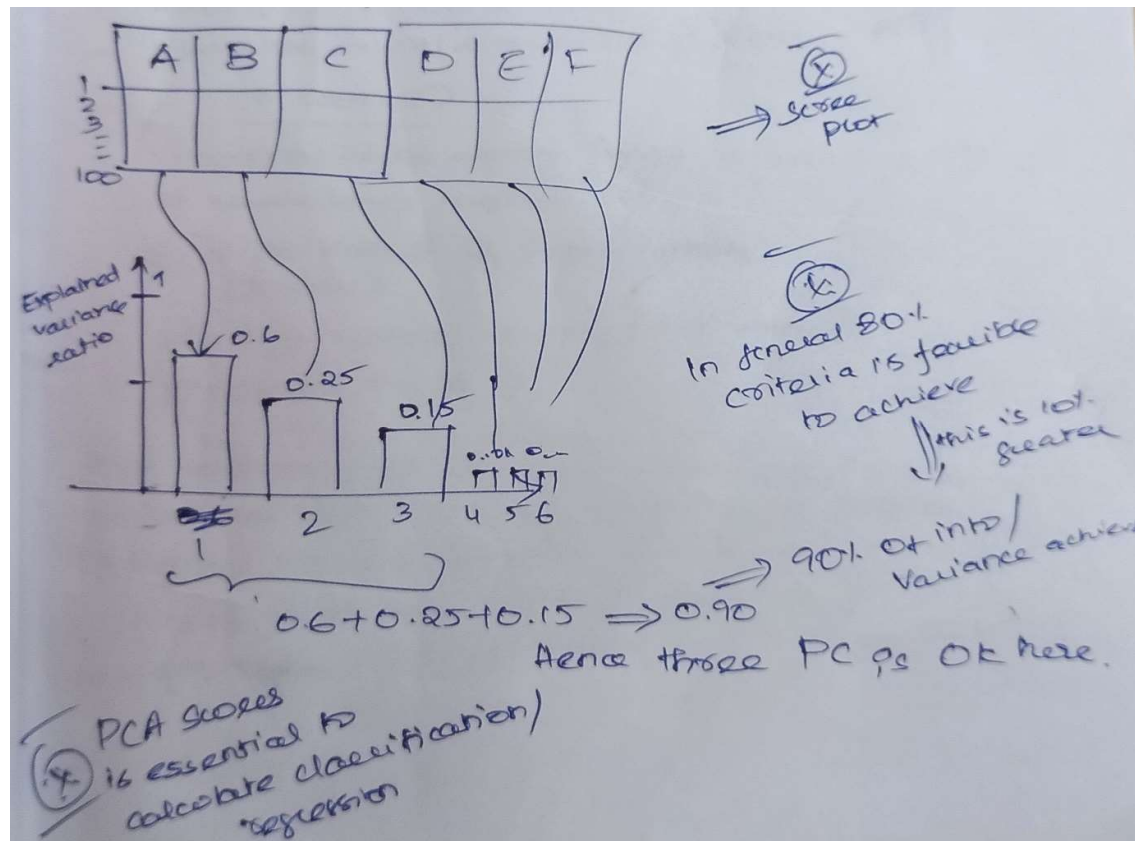
Hence PCA is calculated and visually we can see how PC are orthogonal to each other.

How many PCAs are needed for any data?

This can be understood by,

PCA has maximum variance (information), which will be good to select.

Eigenvalues are used to find out which PCA has a maximum variance.



Advantage for Principal Component Analysis

1. Used for Dimensionality Reduction
2. PCA will assist you in eliminating all related features, sometimes referred to as multi-collinearity.
3. The time required to train your model is now substantially shorter because to PCA's reduction in the number of features.
4. PCA aids in overcoming overfitting by eliminating the extraneous features from your dataset.

Disadvantage for Principal Component Analysis

1. Useful for quantitative data but not effective with qualitative data.
2. Interpretation of PC is difficult from original data

Application for Principal Component Analysis

1. Computer Vision
2. Bio-informatics application
3. For compressed images or resizing of the image
4. Discovering patterns from high-dimensional data
5. Reduction of dimensions
6. Multidimensional Data – Visualization

PCA on Images:

PCA can be applied to images by treating each pixel as a feature and each image as a data point. In this case, the PCA will find the directions in which the pixels vary the most across the set of images, and these directions can be used to represent the images in a lower-dimensional space.

The first step in applying PCA to images is to stack the pixel values of all the images into a single matrix. This matrix will have dimensions $N \times M$, where N is the number of images and M is the number of pixels in each image. Next, we center the data by subtracting the mean value of each pixel across all the images.

The PCA algorithm is then applied to this centered matrix to find the principal components. These principal components are typically ordered in terms of the amount of variance they explain in the data. The first principal component explains the most variance, the second explains the next most, and so on.

Once the principal components have been found, they can be used to represent each image in a lower-dimensional space. To do this, we project each image onto the principal components and retain only the first few components that capture most of the variance in the data. The resulting representation of each image will have a much lower dimensionality than the original pixel representation, but will still retain most of the important information.

PCA is often used as a pre-processing step for other image analysis techniques, such as clustering or classification. By reducing the dimensionality of the data, PCA can make these techniques more efficient and accurate. Additionally, the principal components themselves can be used as a form of image compression, as they capture the most important information in the image in a compact form.

PCA on Olivetti Images:

The Olivetti face dataset is a popular benchmark dataset for face recognition and classification tasks. It consists of 400 grayscale images of faces of 40 individuals, with 10 images per individual, captured under different lighting conditions, facial expressions, and facial details.

PCA can be applied to the Olivetti face dataset to extract the most important features of each face image and reduce the dimensionality of the data. To do this, we first stack the pixel values of all the images into a single matrix, which has dimensions 400×4096 (400 images and 64×64 pixels per image).

Next, we center the data by subtracting the mean value of each pixel across all the images. Then, we perform PCA on the centered matrix to find the principal components. The number of principal components to retain can be chosen based on the amount of variance in the data that they explain. Typically, we retain enough components to capture 90-95% of the variance in the data.

Once the principal components have been found, they can be used to represent each face image in a lower-dimensional space. To do this, we project each face image onto the principal components and retain only the first few components that capture most of the variance in the data. The resulting representation of each face image will have a much lower dimensionality than the original pixel representation, but will still retain most of the important information.

PCA on Olivetti face dataset is often used as a pre-processing step for face recognition or classification tasks, where the low-dimensional representation of each face image is used as input to classification algorithms such as SVM or neural networks. By reducing the dimensionality of the data and capturing the most important features of each face image, PCA can improve the efficiency and accuracy of these algorithms.

Classification of LFW Images:

The Labelled Faces in the Wild (LFW) dataset is a popular benchmark dataset for face recognition and classification tasks. It consists of more than 13,000 face images of 5,749 individuals, captured under a variety of conditions such as illumination, pose, and facial expression.

Classification of LFW images involves assigning each image to the correct identity label. This can be achieved using a variety of machine learning algorithms, such as support vector machines (SVMs), decision trees, random forests, and deep neural networks.

One common approach to classification of LFW images is to use a feature extraction technique such as principal component analysis (PCA) or convolutional neural networks (CNNs) to extract features from each image. These features are then used as input to a classifier to make predictions.

For example, in PCA-based classification, each image is represented as a low-dimensional feature vector obtained by projecting the original high-dimensional pixel values onto a set of principal components. These feature vectors are then used as input to a classifier, such as an SVM or k-nearest neighbors (KNN) algorithm, to make predictions.

In CNN-based classification, a deep neural network is trained on the raw image data to learn a hierarchy of features that are used to classify the images. The network typically consists of multiple layers, with each layer learning increasingly complex and abstract features from the input data.

The performance of classification algorithms on LFW images is typically evaluated using metrics such as accuracy, precision, recall, and F1 score. State-of-the-art methods can achieve accuracy rates of over 99%, indicating that the LFW dataset remains a challenging benchmark for face recognition and classification research.

Association Rule Learning

Association rule learning is a type of unsupervised learning technique that checks for the dependency of one data item on another data item and maps accordingly so that it can be more profitable. It tries to find some interesting relations or associations among the variables of dataset. It is based on different rules to discover the interesting relations between variables in the database.

The association rule learning is one of the very important concepts of **machine learning**, and it is employed in **Market Basket analysis**, **Web usage mining**, **continuous production**, etc. Here market basket analysis is a technique used by the various big retailer to discover the associations between items. We can understand it by taking an example of a supermarket, as in a supermarket, all products that are purchased together are put together.

For example, if a customer buys bread, he most likely can also buy butter, eggs, or milk, so these products are stored within a shelf or mostly nearby. Consider the below diagram:



Association rule learning can be divided into three types of algorithms:

1. Apriori

We will understand these algorithms in later chapters.

How does Association Rule Learning work?

Association rule learning works on the concept of If and Else Statement, such as if A then B.



Here the If element is called **antecedent**, and then statement is called as **Consequent**. These types of relationships where we can find out some association or relation between two items is known as *single cardinality*. It is all about creating rules, and if the number of items increases, then cardinality also increases accordingly. So, to measure the associations between thousands of data items, there are several metrics. These metrics are given below:

- **Support**
- **Confidence**
- **Lift**

Let's understand each of them:

Support

Support is the frequency of A or how frequently an item appears in the dataset. It is defined as the fraction of the transaction T that contains the itemset X. If there are X datasets, then for transactions T, it can be written as:

$$\text{Supp}(X) = \frac{\text{Freq}(X)}{T}$$

Confidence

Confidence indicates how often the rule has been found to be true. Or how often the items X and Y occur together in the dataset when the occurrence of X is already given. It is the ratio of the transaction that contains X and Y to the number of records that contain X.

$$\text{Confidence} = \frac{\text{Freq}(X,Y)}{\text{Freq}(X)}$$

Lift

It is the strength of any rule, which can be defined as below formula:

$$\text{Lift} = \frac{\text{Supp}(X,Y)}{\text{Supp}(X) \times \text{Supp}(Y)}$$

It is the ratio of the observed support measure and expected support if X and Y are independent of each other. It has three possible values:

- If **Lift= 1**: The probability of occurrence of antecedent and consequent is independent of each other.
- **Lift>1**: It determines the degree to which the two item sets are dependent to each other.
- **Lift<1**: It tells us that one item is a substitute for other items, which means one item has a negative effect on another.

Types of Association Rule Learning

Association rule learning can be divided into three algorithms:

Apriori Algorithm

This algorithm uses frequent datasets to generate association rules. It is designed to work on the databases that contain transactions. This algorithm uses a breadth-first search and Hash Tree to calculate the itemset efficiently.

It is mainly used for market basket analysis and helps to understand the products that can be bought together. It can also be used in the healthcare field to find drug reactions for patients.

Applications of Association Rule Learning

It has various applications in machine learning and data mining. Below are some popular applications of association rule learning:

- **Market Basket Analysis**: It is one of the popular examples and applications of association rule mining. This technique is commonly used by big retailers to determine the association between items.

- **Medical Diagnosis:** With the help of association rules, patients can be cured easily, as it helps in identifying the probability of illness for a particular disease.
- **Protein Sequence:** The association rules help in determining the synthesis of artificial Proteins.
- It is also used for the **Catalog Design** and **Loss-leader Analysis** and many more other applications.

Recommendation Engines:

In today's world, every customer is faced with multiple choices. For example, If I'm looking for a book to read without any specific idea of what I want, there's a wide range of possibilities how my search might pan out. I might waste a lot of time browsing around on the internet and trawling through various sites hoping to strike gold. I might look for recommendations from other people.

But if there was a site or app which could recommend me books based on what I have read previously, that would be a massive help. Instead of wasting time on various sites, I could just log in and voila! 10 recommended books tailored to my taste.

This is what recommendation engines do and their power is being harnessed by most businesses these days. From Amazon to Netflix, Google to Goodreads, recommendation engines are one of the most widely used applications of [machine learning techniques](#).

1. What are recommendation engines?

Till recently, people generally tended to buy products recommended to them by their friends or the people they trust. This used to be the primary method of purchase when there was any doubt about the product. But with the advent of the digital age, that circle has expanded to include online sites that utilize some sort of recommendation engine.

A recommendation engine filters the data using different algorithms and recommends the most relevant items to users. It first captures the past

behaviour of a customer and based on that, recommends products which the users might be likely to buy.

If a completely new user visits an e-commerce site, that site will not have any past history of that user. So how does the site go about recommending products to the user in such a scenario? One possible solution could be to recommend the best selling products, i.e. the products which are high in demand. Another possible solution could be to recommend the products which would bring the maximum profit to the business.

If we can recommend a few items to a customer based on their needs and interests, it will create a positive impact on the user experience and lead to frequent visits. Hence, businesses nowadays are building smart and intelligent recommendation engines by studying the past behaviour of their users.

Now that we have an intuition of recommendation engines, let's now look at how they work.

2. How does a recommendation engine work?

Before we deep dive into this topic, first we'll think of how we can recommend items to users:

- We can recommend items to a user which are most popular among all the users
- We can divide the users into multiple segments based on their preferences (user features) and recommend items to them based on the segment they belong to

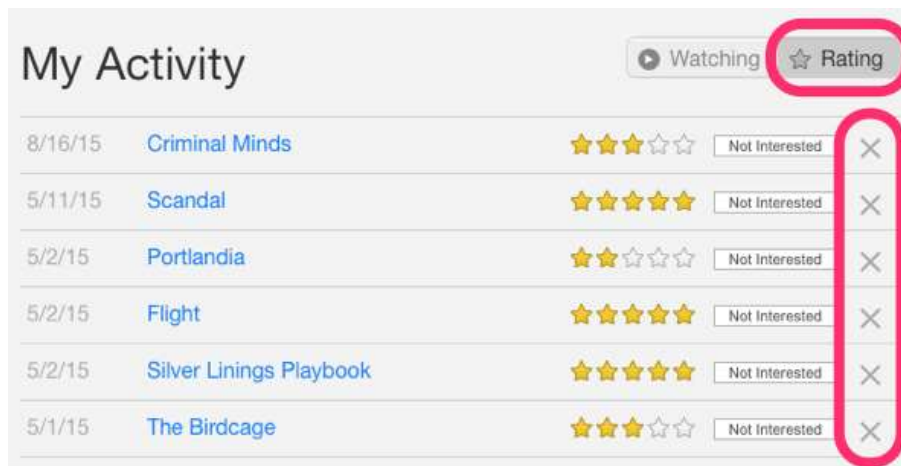
Both of the above methods have their drawbacks. In the first case, the most popular items would be the same for each user so everybody will see the same recommendations. While in the second case, as the number of users increases, the number of features will also increase. So, classifying the users into various segments will be a very difficult task.

The main problem here is that we are unable to tailor recommendations based on the specific interest of the users. It's like Amazon is recommending you buy a laptop just because it's been bought by the majority of the shoppers. But thankfully, Amazon (or any other big firm) does not recommend products using the above mentioned approach. They use some personalized methods which help them in recommending products more accurately.

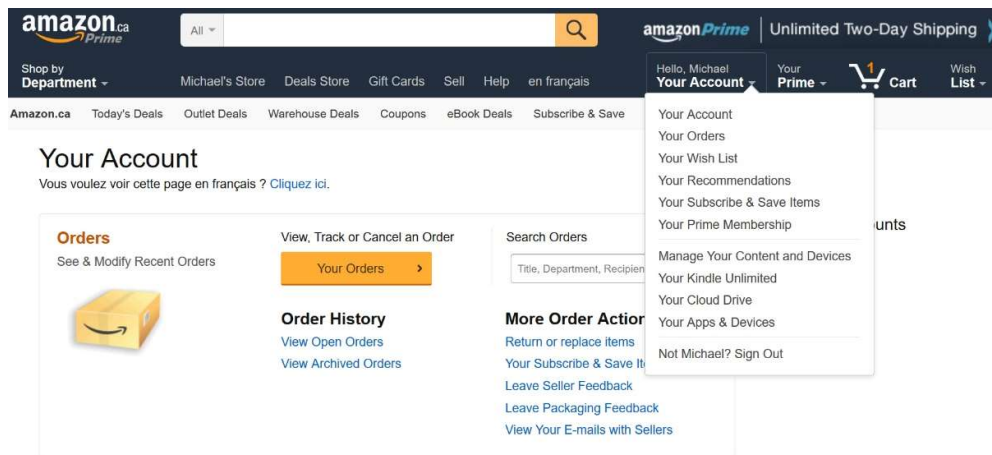
Let's now focus on how a recommendation engine works by going through the following steps.

2.1 Data collection

This is the first and most crucial step for building a recommendation engine. The data can be collected by two means: explicitly and implicitly. Explicit data is information that is provided intentionally, i.e., input from the users such as movie ratings. Implicit data is information that is not provided intentionally but gathered from available data streams like search history, clicks, order history, etc.



In the above image, Netflix is collecting the data explicitly in the form of ratings given by user to different movies.



Here the order history of a user is recorded by Amazon which is an example of implicit mode of data collection.

2.2 Data storage

The amount of data dictates how good the recommendations of the model can get. For example, in a movie recommendation system, the more ratings users give to movies, the better the recommendations get for other users. The type of data plays an important role in deciding the type of storage that has to be used. This type of

storage could include a standard SQL database, a NoSQL database or some kind of object storage.

2.3 Filtering the data

After collecting and storing the data, we have to filter it so as to extract the relevant information required to make the final recommendations.

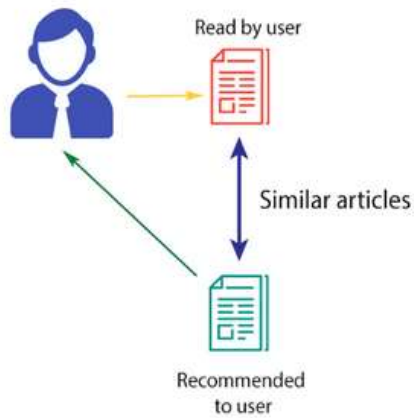


There are various algorithms that help us make the filtering process easier. In the next section, we will go through each algorithm in detail.

2.3.1 *Content based filtering*

This algorithm recommends products which are similar to the ones that a user has liked in the past.

CONTENT-BASED FILTERING



For example, if a person has liked the movie “Inception”, then this algorithm will recommend movies that fall under the same genre. But how does the algorithm understand which genre to pick and recommend movies from?

Consider the example of Netflix. They save all the information related to each user in a vector form. This vector contains the past behaviour of the user, i.e., the movies liked/disliked by the user and the ratings given by them. This vector is known as the *profile vector*. All the information related to movies is stored in another vector called the *item vector*. Item vector contains the details of each movie, like genre, cast, director, etc.

The content-based filtering algorithm finds the cosine of the angle between the profile vector and item vector, i.e., **cosine similarity**. Suppose A is the profile vector and B is the item vector, then the similarity between them can be calculated as:

$$\text{sim}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Based on the cosine value, which ranges between -1 to 1, the movies are arranged in descending order and one of the two below approaches is used for recommendations:

- **Top-n approach:** where the top n movies are recommended (Here n can be decided by the business)
- **Rating scale approach:** Where a threshold is set and all the movies above that threshold are recommended

Other methods that can be used to calculate the similarity are:

- **Euclidean Distance:** Similar items will lie in close proximity to each other if plotted in n-dimensional space. So, we can calculate the distance between items and based on that distance, recommend items to the user. The formula for the Euclidean distance is given by:

$$\text{Euclidean Distance} = \sqrt{(x_1 - y_1)^2 + \dots + (x_N - y_N)^2}$$

- **Pearson's Correlation:** It tells us how much two items are correlated. Higher the correlation, more will be the similarity. Pearson's correlation can be calculated using the following formula:

$$\text{sim}(u, v) = \frac{\sum (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum (r_{ui} - \bar{r}_u)^2} \sqrt{\sum (r_{vi} - \bar{r}_v)^2}}$$

A major drawback of this algorithm is that it is limited to recommending items that are of the same type. It will never recommend products which the user has not bought or liked in the past. So, if a user has watched or liked only action movies in the past, the system will recommend only action movies. It's a very narrow way of building an engine.

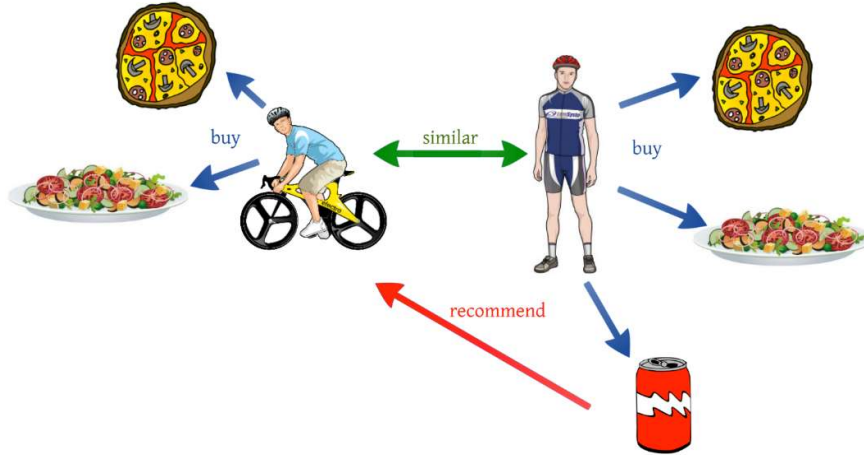
To improve on this type of system, we need an algorithm that can recommend items not just based on the content, but the behaviour of users as well.

2.3.2 Collaborative filtering

Let us understand this with an example. If person A likes 3 movies, say Interstellar, Inception and Predestination, and person B likes Inception, Predestination and The Prestige, then they have almost similar interests. We can say with some certainty that A should like The Prestige and B should like Interstellar. The collaborative filtering algorithm uses “User Behaviour” for recommending items. This is one of the most commonly used algorithms in the industry as it is not dependent on any additional information. There are different types of collaborating filtering techniques and we shall look at them in detail below.

User-User collaborative filtering

This algorithm first finds the similarity score between users. Based on this similarity score, it then picks out the most similar users and recommends products which these similar users have liked or bought previously.



In terms of our movies example from earlier, this algorithm finds the similarity between each user based on the ratings they have previously given to different movies. The prediction of an item for a user u is calculated by computing the weighted sum of the user ratings given by other users to an item i .

The prediction $P_{u,i}$ is given by:

$$P_{u,i} = \frac{\sum_v (r_{v,i} * s_{u,v})}{\sum_v s_{u,v}}$$

Here,

- $P_{u,i}$ is the prediction of an item
- $R_{v,i}$ is the rating given by a user v to a movie i
- $S_{u,v}$ is the similarity between users

Now, we have the ratings for users in profile vector and based on that we have to predict the ratings for other users. Following steps are followed to do so:

1. For predictions we need the similarity between the user u and v . We can make use of Pearson correlation.

2. First, we find the items rated by both the users and based on the ratings, correlation between the users is calculated.
3. The predictions can be calculated using the similarity values. This algorithm, first of all calculates the similarity between each user and then based on each similarity calculates the predictions. **Users having higher correlation will tend to be similar.**
4. Based on these prediction values, recommendations are made. Let us understand it with an example:

Consider the user-movie rating matrix:

User/Movie x1 x2 x3 x4 x5 Mean User Rating

A	4	1	–	4	–	3
B	–	4	–	2	3	3
C	–	1	–	4	4	3

Here we have a user movie rating matrix. To understand this in a more practical manner, let's find the similarity between users (A, C) and (B, C) in the above table. Common movies rated by A/[and C are movies x2 and x4 and by B and C are movies x2, x4 and x5.

$$r_{AC} = [(1-3)*(1-3) + (4-3)*(4-3)]/[(1-3)^2 + (4-3)^2]^{1/2} * [(1-3)^2 + (4-3)^2]^{1/2} = 1$$

$$r_{BC} = [(4-3)*(1-3) + (2-3)*(4-3) + (3-3)*(4-3)]/[(4-3)^2 + (2-3)^2 + (3-3)^2]^{1/2} * [(1-3)^2 + (4-3)^2 + (4-3)^2]^{1/2} = -0.866$$

The correlation between user A and C is more than the correlation between B and C. Hence users A and C have more similarity and the movies liked by user A will be recommended to user C and vice versa.

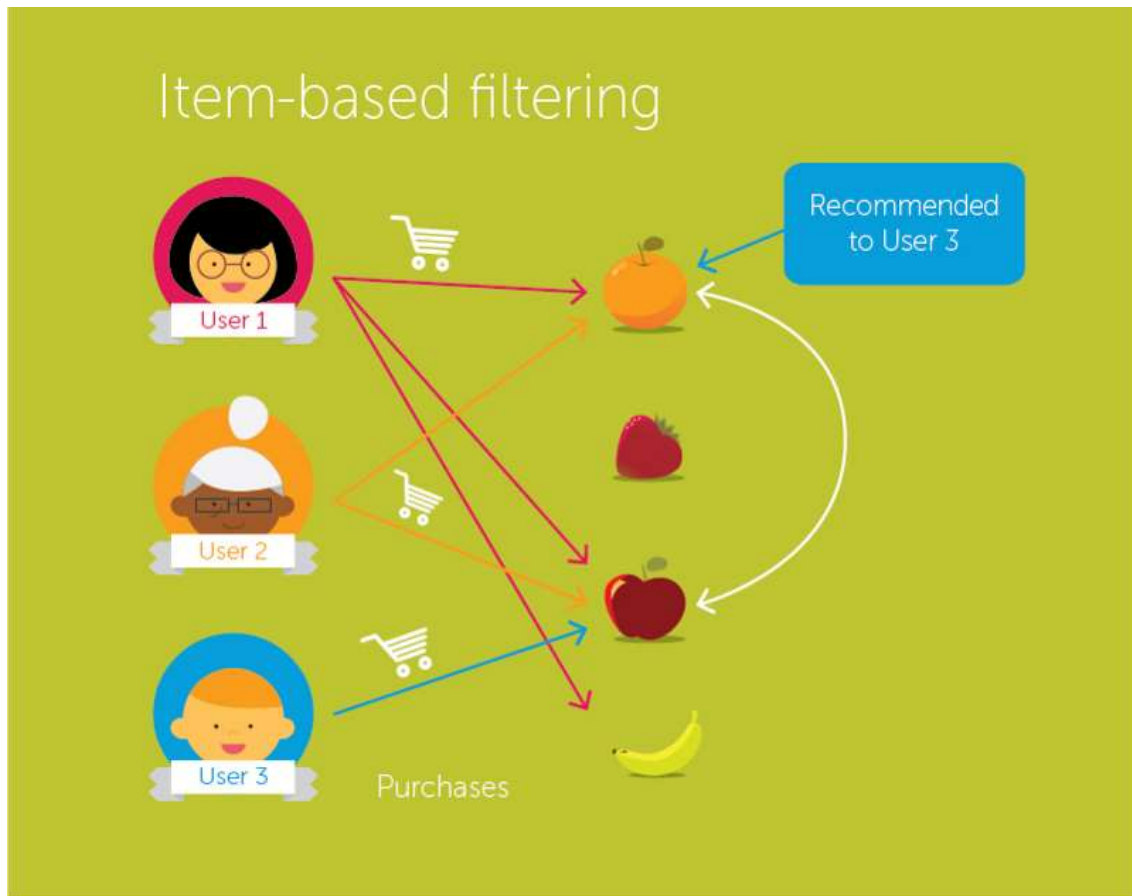
This algorithm is quite time consuming as it involves calculating the similarity for each user and then calculating prediction for each similarity score. One way of handling this problem is to select only a few users (neighbors) instead of all to make predictions, i.e. instead of making predictions for all similarity values, we choose only few similarity values. There are various ways to select the neighbors:

- Select a threshold similarity and choose all the users above that value
- Randomly select the users
- Arrange the neighbors in descending order of their similarity value and choose top-N users
- Use clustering for choosing neighbors

This algorithm is useful when the number of users is less. It's not effective when there are a large number of users as it will take a lot of time to compute the similarity between all user pairs. This leads us to item-item collaborative filtering, which is effective when the number of users is more than the items being recommended.

Item-Item collaborative filtering

In this algorithm, we compute the similarity between each pair of items.



So in our case we will find the similarity between each movie pair and based on that, we will recommend similar movies which are liked by the users in the past. This algorithm works similar to user-user collaborative filtering with just a little change – instead of taking the weighted sum of ratings of “user-neighbors”, we take the weighted sum of ratings of “item-neighbors”. The prediction is given by:

$$P_{u,i} = \frac{\sum_N (s_{i,N} * R_{u,N})}{\sum_N (|s_{i,N}|)}$$

Now we will find the similarity between items.

$$sim(i, j) = \cos(\vec{i}, \vec{j}) = \frac{\vec{i} \cdot \vec{j}}{\|\vec{i}\|_2 * \|\vec{j}\|_2}$$

Now, as we have the similarity between each movie and the ratings, predictions are made and based on those predictions, similar movies are recommended. Let us understand it with an example.

User/Movie	x1	x2	x3	x4	x5
A	4	1	2	4	4
B	2	4	4	2	1
C	–	1	–	3	4
Mean Item Rating	3	2	3	3	3

Here the mean item rating is the average of all the ratings given to a particular item (compare it with the table we saw in user-user filtering). Instead of finding the user-user similarity as we saw earlier, we find the item-item similarity.

To do this, first we need to find such users who have rated those items and based on the ratings, similarity between the items is calculated. Let us find the similarity between movies (x1, x4) and (x1, x5). Common users who have rated movies x1 and x4 are A and B while the users who have rated movies x1 and x5 are also A and B.

$$C_{14} = [(4-3)*(4-3) + (2-3)*(2-3)] / [((4-3)^2 + (2-3)^2)^{1/2} * ((4-3)^2 + (2-3)^2)^{1/2}] = 1$$

$$C_{15} = [(4-3)*(4-3) + (2-3)*(1-3)] / [((4-3)^2 + (2-3)^2)^{1/2} * ((4-3)^2 + (1-3)^2)^{1/2}] = 0.94$$

The similarity between movie x1 and x4 is more than the similarity between movie x1 and x5. So based on these similarity values, if any user searches for movie x1, they will be recommended movie x4 and vice versa. Before going further and implementing these concepts, there is a question which we must know the answer to – what will happen if a new user or a new item is added in the dataset? It is called a **Cold Start**. There can be two types of cold start:

1. Visitor Cold Start
2. Product Cold Start

Visitor Cold Start means that a new user is introduced in the dataset. Since there is no history of that user, the system does not know the preferences of that user. It becomes harder to recommend products to that user. So, how can we solve this problem? One basic approach could be to apply a popularity-based strategy, i.e. recommend the most popular products. These can be determined by what has been popular recently overall or regionally. Once we know the preferences of the user, recommending products will be easier.

On the other hand, Product Cold Start means that a new product is launched in the market or added to the system. User action is most important to determine the value of any product. More the interaction a product receives, the easier it is for our model to recommend that product to the right user. We can make use of Content based filtering to solve this problem. The system first uses the content of the new product for recommendations and then eventually the user actions on that product.

Now let's solidify our understanding of these concepts using a case study in Python. Get your machines ready because this is going to be fun!

3. Case study in Python using the Movie Lens Dataset

We will work on the Movie Lens dataset and build a model to recommend movies to the end users. This data has been collected by the Group Lens Research Project at the University of Minnesota. This dataset consists of:

- 100,000 ratings (1-5) from 943 users on 1682 movies
- Demographic information of the users (age, gender, occupation, etc.)

Content Based Recommender System:

A Content-Based Recommender works by the data that we take from the user, either explicitly (rating) or implicitly (clicking on a link). By the data we create a user profile, which is then used to suggest to the user, as the user provides more input or take more actions on the recommendation, the engine becomes more accurate.

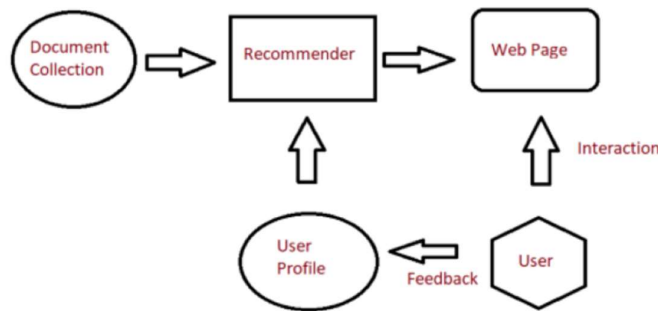


Fig: Recommender System

User Profile:

In the User Profile, we create vectors that describe the user's preference. In the creation of a user profile, we use the utility matrix which describes the relationship between user and item. With this information, the best estimate we can make regarding which item user likes, is some aggregation of the profiles of those items.

Item Profile:

In Content-Based Recommender, we must build a profile for each item, which will represent the important characteristics of that item.

For example, if we make a movie as an item then its actors, director, release year and genre are the most significant features of the movie. We can also add its rating from the IMDB (Internet Movie Database) in the Item Profile.

Utility Matrix:

Utility Matrix signifies the user's preference with certain items. In the data gathered from the user, we have to find some relation between the items which are liked by the user and those which are disliked, for this purpose we use the utility matrix. In it we assign a particular value to each user-item pair, this value

is known as the degree of preference. Then we draw a matrix of a user with the respective items to identify their preference relationship.

Users	Movie 1	Movie 2	Movie 3
User 1	3		1
User 2	2	4	

Fig: Utility Matrix of Movie Recommendation System

Some of the columns are blank in the matrix that is because we don't get the whole input from the user every time, and the goal of a recommendation system is not to fill all the columns but to recommend a movie to the user which he/she will prefer. Through this table, our recommender system won't suggest Movie 3 to User 2, because in Movie 1 they have given approximately the same ratings, and in Movie 3 User 1 has given the low rating, so it is highly possible that User 2 also won't like it.

Recommending Items to User Based on Content:

- **Method 1:**

We can use the cosine distance between the vectors of the item and the user to determine its preference to the user. For explaining this, let us consider an example:

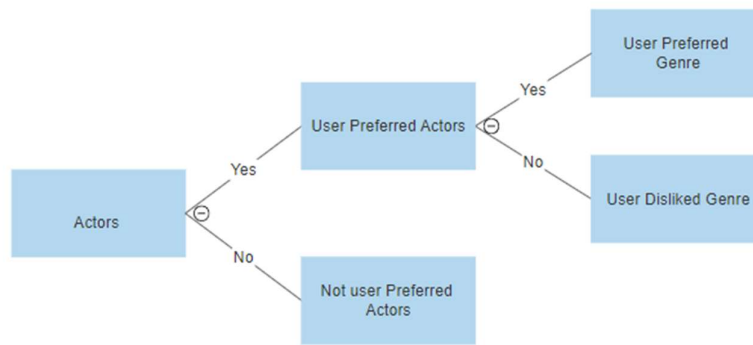
We observe that the vector for a user will have a positive number for actors that tend to appear in movies the user likes and negative numbers for actors user doesn't like, Consider a movie with actors which user likes and only a few actors which user doesn't like, then the cosine angle between the user's and movie's vectors will be a large positive fraction. Thus, the angle will be close to 0, therefore a small cosine distance between the vectors.

It represents that the user tends to like the movie, if the cosine distance is large, then we tend to avoid the item from the recommendation.

- **Method 2:**

We can use a classification approach in the recommendation systems too, like we can use the Decision Tree for finding out whether a user wants to watch a movie or not, like at each level we can apply a certain condition to refine our recommendation.

For example:

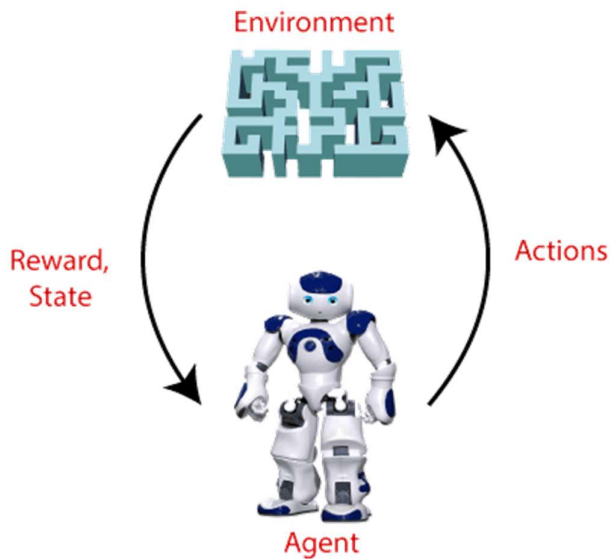


Reinforcement Learning

What is Reinforcement Learning?

- Reinforcement Learning is a feedback-based Machine learning technique in which an agent learns to behave in an environment by performing the actions and seeing the results of actions. For each good action, the agent gets positive feedback, and for each bad action, the agent gets negative feedback or penalty.
- In Reinforcement Learning, the agent learns automatically using feedbacks without any labelled data, unlike **supervised learning**.
- Since there is no labelled data, so the agent is bound to learn by its experience only.
- RL solves a specific type of problem where decision making is sequential, and the goal is long-term, such as **game-playing, robotics**, etc.
- The agent interacts with the environment and explores it by itself. The primary goal of an agent in reinforcement learning is to improve the performance by getting the maximum positive rewards.
- The agent learns with the process of hit and trial, and based on the experience, it learns to perform the task in a better way. Hence, we can say that *"Reinforcement learning is a type of machine learning method where an intelligent agent (computer program) interacts with the environment and learns to act within that."* How a Robotic dog learns the movement of his arms is an example of Reinforcement learning.
- It is a core part of **Artificial intelligence**, and all **AI agent** works on the concept of reinforcement learning. Here we do not need to pre-program the agent, as it learns from its own experience without any human intervention.
- **Example:** Suppose there is an AI agent present within a maze environment, and his goal is to find the diamond. The agent interacts with the environment by performing some actions, and based on those actions, the state of the agent gets changed, and it also receives a reward or penalty as feedback.
- The agent continues doing these three things (**take action, change state/remain in the same state, and get feedback**), and by doing these actions, he learns and explores the environment.

- The agent learns that what actions lead to positive feedback or rewards and what actions lead to negative feedback penalty. As a positive reward, the agent gets a positive point, and as a penalty, it gets a negative point.



Terms used in Reinforcement Learning

- **Agent():** An entity that can perceive/explore the environment and act upon it.
- **Environment():** A situation in which an agent is present or surrounded by. In RL, we assume the stochastic environment, which means it is random in nature.
- **Action():** Actions are the moves taken by an agent within the environment.
- **State():** State is a situation returned by the environment after each action taken by the agent.
- **Reward():** A feedback returned to the agent from the environment to evaluate the action of the agent.
- **Policy():** Policy is a strategy applied by the agent for the next action based on the current state.
- **Value():** It is expected long-term return with the discount factor and opposite to the short-term reward.
- **Q-value():** It is mostly similar to the value, but it takes one additional parameter as a current action (a).

Key Features of Reinforcement Learning

- In RL, the agent is not instructed about the environment and what actions need to be taken.
- It is based on the hit and trial process.
- The agent takes the next action and changes states according to the feedback of the previous action.
- The agent may get a delayed reward.
- The environment is stochastic, and the agent needs to explore it to reach to get the maximum positive rewards.

Approaches to implement Reinforcement Learning

There are mainly three ways to implement reinforcement-learning in ML, which are:

1. **Value-based:**

The value-based approach is about to find the optimal value function, which is the maximum value at a state under any policy. Therefore, the agent expects the long-term return at any state(s) under policy π .

2. **Policy-based:**

Policy-based approach is to find the optimal policy for the maximum future rewards without using the value function. In this approach, the agent tries to apply such a policy that the action performed in each step helps to maximize the future reward.

The policy-based approach has mainly two types of policy:

- **Deterministic:** The same action is produced by the policy (π) at any state.
 - **Stochastic:** In this policy, probability determines the produced action.
- ### 3. **Model-based:** In the model-based approach, a virtual model is created for the environment, and the agent explores that environment to learn it. There is no particular solution or algorithm for this approach because the model representation is different for each environment.

Elements of Reinforcement Learning

There are four main elements of Reinforcement Learning, which are given below:

1. Policy
2. Reward Signal
3. Value Function
4. Model of the environment

1) Policy: A policy can be defined as a way how an agent behaves at a given time. It maps the perceived states of the environment to the actions taken on those states. A policy is the core element of the RL as it alone can define the behaviour of the agent. In some cases, it may be a simple function or a lookup table, whereas, for other cases, it may involve general computation as a search process. It could be deterministic or a stochastic policy:

For deterministic policy: $a = \pi(s)$

For stochastic policy: $\pi(a | s) = P[A_t = a | S_t = s]$

2) Reward Signal: The goal of reinforcement learning is defined by the reward signal. At each state, the environment sends an immediate signal to the learning agent, and this signal is known as a **reward signal**. These rewards are given according to the good and bad actions taken by the agent. The agent's main objective is to maximize the total number of rewards for good actions. The reward signal can change the policy, such as if an action selected by the agent leads to low reward, then the policy may change to select other actions in the future.

3) Value Function: The value function gives information about how good the situation and action are and how much reward an agent can expect. A reward indicates the **immediate signal for each good and bad action**, whereas a value function specifies **the good state and action for the future**. The value function depends on the reward as, without reward, there could be no value. The goal of estimating values is to achieve more rewards.

4) Model: The last element of reinforcement learning is the model, which mimics the behaviour of the environment. With the help of the model, one can make inferences about how the environment will behave. Such as, if a state and an action are given, then a model can predict the next state and reward.

The model is used for planning, which means it provides a way to take a course of action by considering all future situations before actually experiencing those situations. The approaches for solving the RL problems **with the help of the**

model are termed as the **model-based approach**. Comparatively, an approach **without using a model** is called a **model-free approach**.

What are Exploration and Exploitation in Reinforcement Learning

Before going to a brief description of exploration and exploitation in machine learning, let's first understand these terms in simple words. In reinforcement learning, whenever agents get a situation in which they have to make a difficult choice between whether to continue the same work or explore something new at a specific time, then, this situation results in Exploration-Exploitation Dilemma because the knowledge of an agent about the state, actions, rewards and resulting states is always partial.

Now we will discuss exploitation and exploration in technical terms.

Exploitation in Reinforcement Learning

Exploitation is defined as a greedy approach in which agents try to get more rewards by using estimated value but not the actual value. So, in this technique, *agents make the best decision based on current information.*

Exploration in Reinforcement Learning

Unlike exploitation, in exploration techniques, agents primarily focus on improving their knowledge about each action instead of getting more rewards so that they can get long-term benefits. So, in this technique, *agents work on gathering more information to make the best overall decision.*

Examples of Exploitation and Exploration in Machine Learning

Let's understand exploitation and exploration with some interesting real-world examples.

Coal mining:

Let's suppose people A and B are digging in a coal mine in the hope of getting a diamond inside it. Person B got success in finding the diamond before person A and walks off happily. After seeing him, person A gets a bit greedy and thinks he too might get success in finding diamond at the same place where person B was digging coal. This action performed by person A is called **greedy action**, and this policy is known as **a greedy policy**. But person A was unknown because a bigger diamond was buried in that place where he was initially digging the coal, and this greedy policy would fail in this situation.

In this example, person A only got knowledge of the place where person B was digging but had no knowledge of what lies beyond that depth. But in the actual scenario, the diamond can also be buried in the same place where he was digging

initially or some completely another place. Hence, with this partial knowledge about getting more rewards, our reinforcement learning agent will be in a dilemma on whether to exploit the partial knowledge to receive some rewards or it should explore unknown actions which could result in many rewards.

However, both these techniques are not feasible simultaneously, but this issue can be resolved by using Epsilon Greedy Policy (Explained below).

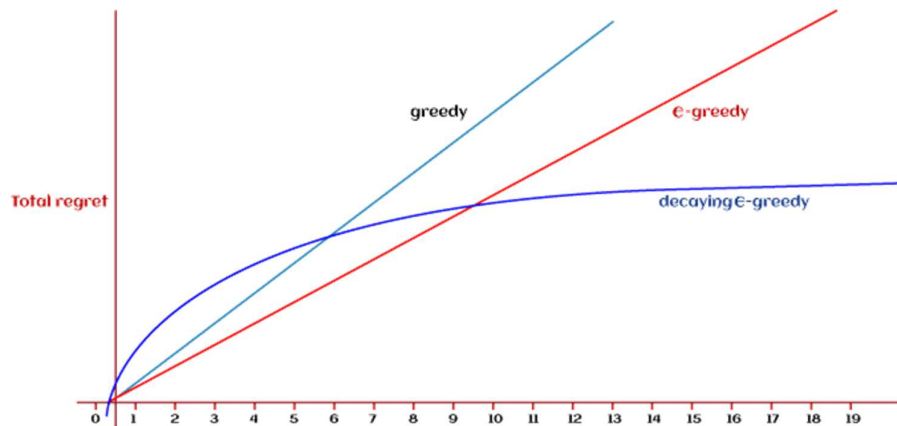
There are a few other examples of Exploitation and Exploration in Machine Learning as follows:

Example 1: Let's say we have a scenario of online restaurant selection for food orders, where you have two options to select the restaurant. In the first option, you can choose your favourite restaurant from where you ordered food in the past; this is called **exploitation** because here, you only know information about a specific restaurant. And for other options, you can try a new restaurant to explore new varieties and tastes of food, and it is called exploration. However, food quality might be better in the first option, but it is also possible that it is more delicious in another restaurant.

Example 2: Suppose there is a game-playing platform where you can play chess with robots. To win this game, you have two choices either play the move that you believe is best, and for the other choice, you can play an experimental move. However, you are playing the best possible move, but who knows new move might be more strategic to win this game. Here, the first choice is called exploitation, where you know about your game strategy, and the second choice is called exploration, where you are exploring your knowledge and playing a new move to win the game.

Epsilon Greedy Policy

Epsilon greedy policy is defined as a technique to maintain a balance between exploitation and exploration. However, to choose between exploration and exploitation, a very simple method is to select randomly. This can be done by choosing exploitation most of the time with a little exploration.



In the greedy epsilon strategy, an exploration rate or epsilon (denoted as ϵ) is initially set to 1. **This exploration rate defines the probability of exploring the environment by the agent rather than exploiting it.** It also ensures that the agent will start by exploring the environment with $\epsilon=1$.

As the agent starts and learns more about the environment, the epsilon decreases by some rate in the defined rate, so the likelihood of exploration becomes less and less probable as the agent learns more and more about the environment. In such a case, the agent becomes greedy for exploiting the environment.

To find if the agent will select exploration or exploitation at each step, we generate a random number between 0 and 1 and compare it to the epsilon. If this random number is greater than ϵ , then the next action would be decided by the exploitation method. Else it must be exploration. In the case of exploitation, the agent will take action with the highest Q-value for the current state.

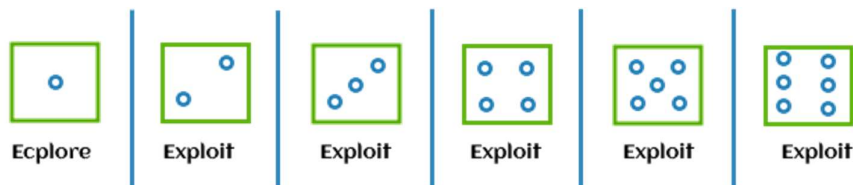
1. if random_number > epsilon:
2. //choose next action via exploitation
3. else:
4. // choose next action via exploration

Examples-

We can understand the above concept with rolling dice. Let's say the agent will explore if the dice land on 1; otherwise, he will exploit. This method is called an epsilon greedy action with the value of epsilon $\epsilon=1/6$, which is the probability of getting 1 on dice. It can be expressed as follows:

$$A_t \leftarrow \begin{cases} \operatorname{argmax} Q_t(a) \\ a \sim \operatorname{Unifrom}(\{a_1 \dots a_k\}) \end{cases} \quad \begin{array}{l} \text{with probability } 1 - \epsilon \\ \text{with probability } \epsilon \end{array}$$

In the above formula, the action selected at attempt 't' will be a greedy action (exploit) with probability $1 - \epsilon$ or maybe a random action (explore) with probability ϵ .



Notion of Regret

Whenever we do something and don't find the proper outcome, then regret our decision as we have previously discussed an example of exploitation and exploration for choosing a restaurant. For that example, if we choose a new restaurant instead of our favourite, but the food quality and overall experience are poor, then we will regret our decision and will consider what we paid for as a complete loss. Moreover, if we order food from the same restaurant again, the regret level increases along with the number of losses. However, reinforcement learning methods can reduce the amount of loss and the level of regret.

Regret in Reinforcement Learning

Before understanding the regret in reinforcement learning, we must know the optimal action ' a^* ', which is the action that gives the highest rewards. It is given as follows:

$$a^* = \max_{a \in A} E[r | a]$$

Hence, the regret in reinforcement learning can be defined as the difference between the reward generated by the optimal action a^* multiplied by T and the sum from 1 to T of each reward of arbitrary action. It can be expressed as follows:

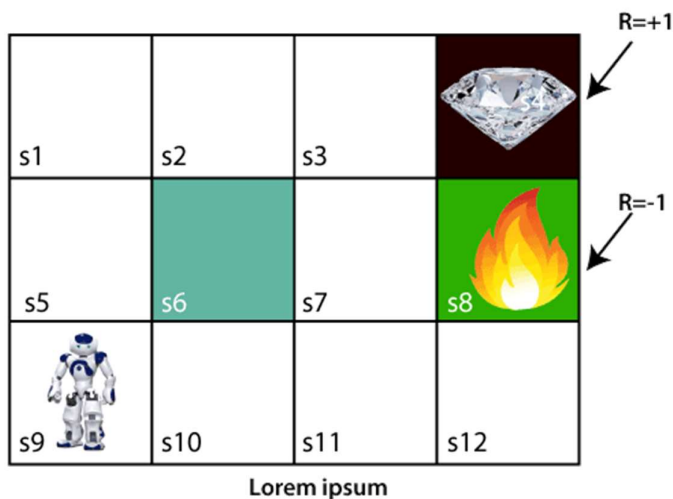
$$\text{Regret: } L_T = TE[r^{a^*}] - \sum [r|a_t]$$

How does Reinforcement Learning Work?

To understand the working process of the RL, we need to consider two main things:

- **Environment:** It can be anything such as a room, maze, football ground, etc.
- **Agent:** An intelligent agent such as AI robot.

Let's take an example of a maze environment that the agent needs to explore. Consider the below image:



In the above image, the agent is at the very first block of the maze. The maze is consisting of an S_6 block, which is a **wall**, S_8 a **fire pit**, and S_4 a **diamond block**.

The agent cannot cross the S_6 block, as it is a solid wall. If the agent reaches the S_4 block, then get the **+1 reward**; if it reaches the fire pit, then gets **-1 reward point**. It can take four actions: **move up, move down, move left, and move right**.

The agent can take any path to reach to the final point, but he needs to make it in possible fewer steps. Suppose the agent considers the path **S9-S5-S1-S2-S3**, so he will get the +1-reward point.

The agent will try to remember the preceding steps that it has taken to reach the final step. To memorize the steps, it assigns 1 value to each previous step. Consider the below step:

$V=1$ s1	$V=1$ s2	$V=1$ s3	 s4
$V=1$ s5		s7	 s8
 $V=1$ s9	s10	s11	s12

Now, the agent has successfully stored the previous steps assigning the 1 value to each previous block. But what will the agent do if he starts moving from the block, which has 1 value block on both sides? Consider the below diagram:

 s1	 $V=1$ s2	$V=1$ s3	 s4
 $V=1$ s5		s7	 s8
$V=1$ s9	s10	s11	s12

It will be a difficult condition for the agent whether he should go up or down as each block has the same value. So, the above approach is not suitable for the agent to reach the destination. Hence to solve the problem, we will use the **Bellman equation**, which is the main concept behind reinforcement learning.

The Bellman Equation

The Bellman equation was introduced by the Mathematician **Richard Ernest Bellman in the year 1953**, and hence it is called as a Bellman equation. It is associated with dynamic programming and used to calculate the values of a decision problem at a certain point by including the values of previous states.

It is a way of calculating the value functions in dynamic programming or environment that leads to modern reinforcement learning.

The key-elements used in Bellman equations are:

- Action performed by the agent is referred to as "a"
- State occurred by performing the action is "s."
- The reward/feedback obtained for each good and bad action is "R."
- A discount factor is Gamma " γ ."

The Bellman equation can be written as:

$$1. V(s) = \max [R(s,a) + \gamma V(s')]$$

Where,

$V(s)$ = value calculated at a particular point.

$R(s,a)$ = Reward at a particular states by performing an action.

γ = Discount factor

$V(s')$ = The value at the previous state.

In the above equation, we are taking the max of the complete values because the agent tries to find the optimal solution always.

So now, using the Bellman equation, we will find value at each state of the given environment. We will start from the block, which is next to the target block.

For 1st block:

$V(s_3) = \max [R(s,a) + \gamma V(s')]$, here $V(s') = 0$ because there is no further state to move.

$V(s_3) = \max[R(s,a)] \Rightarrow V(s_3) = \max[1] \Rightarrow V(s_3) = 1.$

For 2nd block:

$V(s_2) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 1$, and $R(s, a) = 0$, because there is no reward at this state.

$$V(s_2) = \max[0.9(1)] \Rightarrow V(s) = \max[0.9] \Rightarrow \mathbf{V(s_2) = 0.9}$$

For 3rd block:

$V(s_1) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.9$, and $R(s, a) = 0$, because there is no reward at this state also.

$$V(s_1) = \max[0.9(0.9)] \Rightarrow V(s_3) = \max[0.81] \Rightarrow \mathbf{V(s_1) = 0.81}$$

For 4th block:

$V(s_5) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.81$, and $R(s, a) = 0$, because there is no reward at this state also.

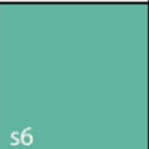
$$V(s_5) = \max[0.9(0.81)] \Rightarrow V(s_5) = \max[0.73] \Rightarrow \mathbf{V(s_5) = 0.73}$$

For 5th block:

$V(s_9) = \max [R(s,a) + \gamma V(s')]$, here $\gamma = 0.9$ (lets), $V(s') = 0.73$, and $R(s, a) = 0$, because there is no reward at this state also.

$$V(s_9) = \max[0.9(0.73)] \Rightarrow V(s_4) = \max[0.66] \Rightarrow \mathbf{V(s_4) = 0.66}$$

Consider the below image:

$V=0.81$ s1	$V=0.9$ s2	$V=1$ s3	 s4
$V=0.73$ s5	 s6	s7	 s8
 $V=0.66$ s9	s10	s11	s12

Now, we will move further to the 6th block, and here agent may change the route because it always tries to find the optimal path. So now, let's consider from the block next to the fire pit.

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5	 s6	 s7	 s8
V=0.66 s9			

Now, the agent has three options to move; if he moves to the blue box, then he will feel a bump if he moves to the fire pit, then he will get the -1 reward. But here we are taking only positive rewards, so for this, he will move to upwards only. The complete block values will be calculated using this formula. Consider the below image:

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5	 s6	V=0.9 s7	 s8
V=0.66 s9	V=0.73 s10	V=0.81 s11	V=0.73 s12

Types of Reinforcement learning

There are mainly two types of reinforcement learning, which are:

- **Positive Reinforcement**
- **Negative Reinforcement**

Positive Reinforcement:

The positive reinforcement learning means adding something to increase the tendency that expected behaviour would occur again. It impacts positively on the behaviour of the agent and increases the strength of the behaviour.

This type of reinforcement can sustain the changes for a long time, but too much positive reinforcement may lead to an overload of states that can reduce the consequences.

Negative Reinforcement:

The negative reinforcement learning is opposite to the positive reinforcement as it increases the tendency that the specific behaviour will occur again by avoiding the negative condition.

It can be more effective than the positive reinforcement depending on situation and behaviour, but it provides reinforcement only to meet minimum behaviour.

How to represent the agent state?

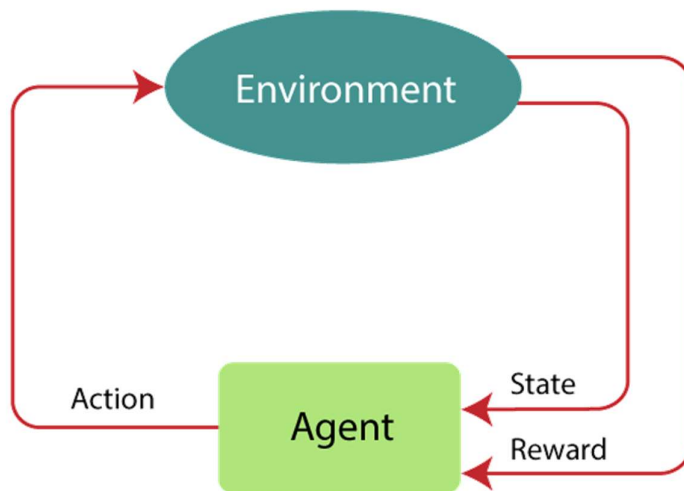
We can represent the agent state using the **Markov State** that contains all the required information from the history. The State S_t is Markov state if it follows the given condition:

$$P[S_{t+1} | S_t] = P[S_{t+1} | S_1, \dots, S_t]$$

The Markov state follows the **Markov property**, which says that the future is independent of the past and can only be defined with the present. The RL works on fully observable environments, where the agent can observe the environment and act for the new state. The complete process is known as Markov Decision process, which is explained below:

Markov Decision Process

Markov Decision Process or MDP, is used to **formalize the reinforcement learning problems**. If the environment is completely observable, then its dynamic can be modelled as a **Markov Process**. In MDP, the agent constantly interacts with the environment and performs actions; at each action, the environment responds and generates a new state.



MDP is used to describe the environment for the RL, and almost all the RL problem can be formalized using MDP.

MDP contains a tuple of four elements (S , A , P_a , R_a):

- A set of finite States S
- A set of finite Actions A
- Rewards received after transitioning from state S to state S' , due to action a .
- Probability P_a .

MDP uses **Markov property**, and to better understand the MDP, we need to learn about it.

Markov Property:

It says that *"If the agent is present in the current state S_1 , performs an action a_1 and move to the state s_2 , then the state transition from s_1 to s_2 only depends on the current state and future action and states do not depend on past actions, rewards, or states."*

Or, in other words, as per Markov Property, the current state transition does not depend on any past action or state. Hence, MDP is an RL problem that satisfies the Markov property. Such as in a **Chess game, the players only focus on the current state and do not need to remember past actions or states.**

Finite MDP:

A finite MDP is when there are finite states, finite rewards, and finite actions. In RL, we consider only the finite MDP.

Markov Process:

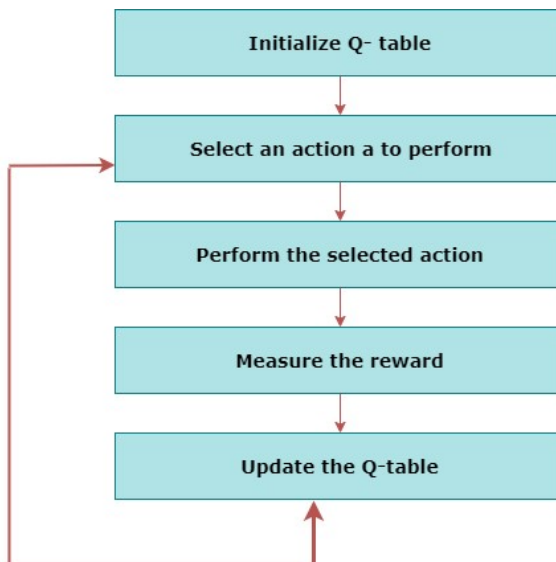
Markov Process is a memoryless process with a sequence of random states $S_1, S_2, \dots, S_t$ that uses the Markov Property. Markov process is also known as Markov chain, which is a tuple (S, P) on state S and transition function P . These two components (S and P) can define the dynamics of the system.

Reinforcement Learning Algorithms

Reinforcement learning algorithms are mainly used in AI applications and gaming applications. The main used algorithms are:

- **Q-Learning:**

- Q-learning is an **Off-policy RL algorithm**, which is used for the temporal difference Learning. The temporal difference learning methods are the way of comparing temporally successive predictions.
- It learns the value function $Q(S, a)$, which means how good to take action "**a**" at a particular state "**s**."
- The below flowchart explains the working of Q- learning:



- **State Action Reward State action (SARSA):**

- SARSA stands for **State Action Reward State action**, which is an **on-policy** temporal difference learning method. The on-policy control method selects the action for each state while learning using a specific policy.
- The goal of SARSA is to calculate the $Q \pi(s, a)$ for the selected **current policy π and all pairs of (s-a)**.
- The main difference between Q-learning and SARSA algorithms is that **unlike Q-learning, the maximum reward for the next state is not required for updating the Q-value in the table.**

- In SARSA, new action and reward are selected using the same policy, which has determined the original action.
- The SARSA is named because it uses the quintuple **Q(s, a, r, s', a')**.

Where,

s: original state

a: Original action

r: reward observed while following the states

s' and a': New state, action pair.

- **Deep Q Neural Network (DQN):**
 - As the name suggests, DQN is a **Q-learning using Neural networks**.
 - For a big state space environment, it will be a challenging and complex task to define and update a Q-table.
 - To solve such an issue, we can use a DQN algorithm. Where, instead of defining a Q-table, neural network approximates the Q-values for each action and state.

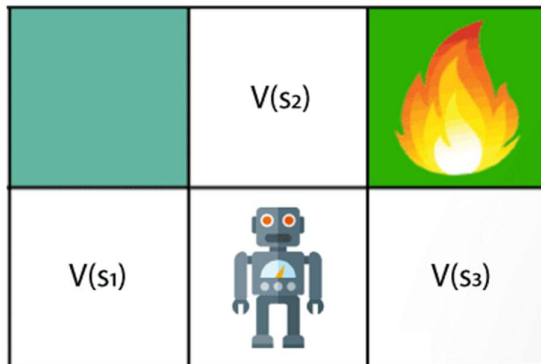
Now, we will expand the Q-learning.

Q-Learning Explanation:

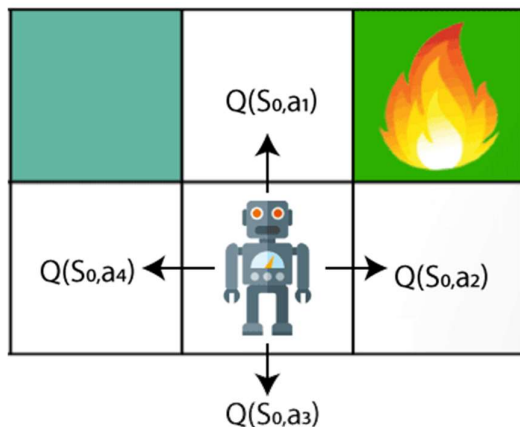
- Q-learning is a popular model-free reinforcement learning algorithm based on the Bellman equation.
- **The main objective of Q-learning is to learn the policy which can inform the agent that what actions should be taken for maximizing the reward under what circumstances.**
- It is an **off-policy RL** that attempts to find the best action to take at a current state.
- The goal of the agent in Q-learning is to maximize the value of Q.
- The value of Q-learning can be derived from the Bellman equation. Consider the Bellman equation given below:

$$V(s) = \max [R(s,a) + \gamma \sum_{s'} P(s, a, s') V(s')]$$

In the equation, we have various components, including reward, discount factor (γ), probability, and end states s' . But there is no any Q-value is given so first consider the below image:



In the above image, we can see there is an agent who has three values options, $V(s_1)$, $V(s_2)$, $V(s_3)$. As this is MDP, so agent only cares for the current state and the future state. The agent can go to any direction (Up, Left, or Right), so he needs to decide where to go for the optimal path. Here agent will take a move as per probability bases and changes the state. But if we want some exact moves, so for this, we need to make some changes in terms of Q-value. Consider the below image:



Q- represents the quality of the actions at each state. So instead of using a value at each state, we will use a pair of state and action, i.e., $Q(s, a)$. Q-value specifies that which action is more lubricative than others, and according to the best Q-value, the agent takes his next move. The Bellman equation can be used for deriving the Q-value.

To perform any action, the agent will get a reward $R(s, a)$, and also, he will end up on a certain state, so the Q -value equation will be:

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} P(s, a, s') V(s')$$

Hence, we can say that, $V(s) = \max [Q(s, a)]$

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} (P(s, a, s') \max_{a'} Q(s', a'))$$

The above formula is used to estimate the Q-values in Q-Learning.

What is 'Q' in Q-learning?

The Q stands for **quality** in **Q-learning**, which means it specifies the quality of an action taken by the agent.

Q-table:

A Q-table or matrix is created while performing the Q-learning. The table follows the state and action pair, i.e., [s, a], and initializes the values to zero. After each action, the table is updated, and the q-values are stored within the table.

The RL agent uses this Q-table as a reference table to select the best action based on the q-values.

Difference between Reinforcement Learning and Supervised Learning

Reinforcement Learning	Supervised Learning
RL works by interacting with the environment.	Supervised learning works on the existing dataset.
The RL algorithm works like the human brain works when making some decisions.	Supervised Learning works as when a human learns things in the supervision of a guide.
There is no labelled dataset is present	The labelled dataset is present.
No previous training is provided to the learning agent.	Training is provided to the algorithm so that it can predict the output.
RL helps to take decisions sequentially.	In Supervised learning, decisions are made when input is given.

Reinforcement Learning Applications

1. **Robotics:**

- a. RL is used in **Robot navigation, Robo-soccer, walking, juggling**, etc.

2. **Control:**

- a. RL can be used for **adaptive control** such as Factory processes, admission control in telecommunication, and Helicopter pilot is an example of reinforcement learning.

3. **Game Playing:**

- a. RL can be used in **Game playing** such as tic-tac-toe, chess, etc.

4. **Chemistry:**

- a. RL can be used for optimizing the chemical reactions.

5. **Business:**

- a. RL is now used for business strategy planning.

6. **Manufacturing:**

- a. In various automobile manufacturing companies, the robots use deep reinforcement learning to pick goods and put them in some containers.

7. **Finance Sector:**

- a. The RL is currently used in the finance sector for evaluating trading strategies.

Time Series Analysis

What Is Time Series Analysis?

Definition: If you see, there are many more definitions for TSA. But let's keep it simple.

A time series is nothing but a sequence of various data points that occurred in a successive order for a given period of time

Objectives of Time Series Analysis:

- To understand how time series works and what factors affect a certain variable(s) at different points in time.
- Time series analysis will provide the consequences and insights of the given dataset's features that change over time.
- Supporting to derive the predicting the future values of the time series variable.
- Assumptions: There is only one assumption in TSA, which is “stationary,” which means that the origin of time does not affect the properties of the process under the statistical factor.

How to Analyse Time Series?

To perform the time series analysis, we have to follow the following steps:

- Collecting the data and cleaning it
- Preparing Visualization with respect to time vs key feature
- Observing the stationarity of the series
- Developing charts to understand its nature.
- Model building – AR, MA, ARMA and ARIMA
- Extracting insights from prediction

Significance of Time Series

TSA is the backbone for prediction and forecasting analysis, specific to time-based problem statements.

- Analysing the historical dataset and its patterns
- Understanding and matching the current situation with patterns derived from the previous stage.
- Understanding the factor or factors influencing certain variable(s) in different periods.

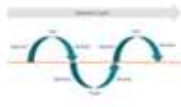
With the help of “Time Series,” we can prepare numerous time-based analyses and results.

- **Forecasting:** Predicting any value for the future.
- **Segmentation:** Grouping similar items together.
- **Classification:** Classifying a set of items into given classes.
- **Descriptive analysis:** Analysis of a given dataset to find out what is there in it.
- **Intervention analysis:** Effect of changing a given variable on the outcome.

Components of Time Series Analysis

Let's look at the various components of Time Series Analysis-

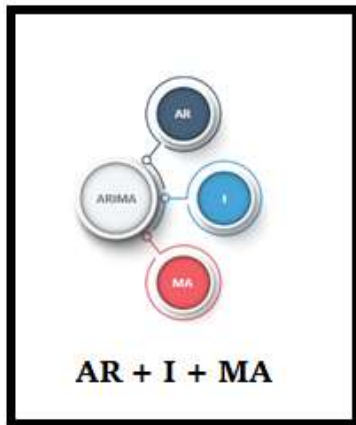
- **Trend:** In which there is no fixed interval and any divergence within the given dataset is a continuous timeline. The trend would be Negative or Positive or Null Trend
- **Seasonality:** In which regular or fixed interval shifts within the dataset in a continuous timeline. Would be bell curve or saw tooth
- **Cyclical:** In which there is no fixed interval, uncertainty in movement and its pattern
- **Irregularity:** Unexpected situations/events/scenarios and spikes in a short time span.

	Trend	Seasonality	Cyclical	Irregularity
Time	Fixed Time Interval	Fixed Time Interval	Not Fixed Time Interval	Not Fixed Time Interval
Duration	Long and Short Term	Short Term	Long and Short Term	Regular/Irregular
Visualization				
Nature - I	Gradual	Swings between Up or Down	Repeating Up and Down	Errored or High Fluctuation
Nature – II	Upward/Down Trend	Pattern repeatable	No fixed period	Short and Not repeatable
Prediction Capability	Predictable	Predictable	Challenging	Challenging
Market Model				Highly random/Unforeseen Events – along with white noise.

Understanding ARMA and ARIMA

ARMA is a combination of the Auto-Regressive and Moving Average models for forecasting. This model provides a weakly stationary stochastic process in terms of two polynomials, one for the Auto-Regressive and the second for the Moving Average.

ARMA is best for predicting stationary series. **ARIMA** was thus developed to support both stationary as well as non-stationary series.



- AR ==> Uses past values to predict the future.
- MA ==> Uses past error terms in the given series to predict the future.
- I==> Uses the differencing of observation and makes the stationary data.

AR+I+MA= ARIMA

Understand the signature of ARIMA

- p ==> lag order ==> No of lag observations.
- d ==> degree of differencing ==> No of times that the raw observations are differenced.

- $q \Rightarrow$ order of moving average $\Rightarrow$ the size of the moving average window

Implementation Steps for ARIMA

Step 1: Plot a time series format

Step 2: Difference to make stationary on mean by removing the trend

Step 3: Make stationary by applying log transform.

Step 4: Difference log transform to make as stationary on both statistic mean and variance

Step 5: Plot ACF & PACF, and identify the potential AR and MA model

Step 6: Discovery of best fit ARIMA model

Step 7: Forecast/Predict the value using the best fit ARIMA model

Step 8: Plot ACF & PACF for residuals of the ARIMA model, and ensure no more information is left.

Stationarity

Methods to Check Stationarity

During the TSA model preparation workflow, we must assess whether the dataset is stationary or not. This is done using **Statistical Tests**. There are two tests available to test if the dataset is stationary:

- Augmented Dickey-Fuller (ADF) Test
- Kwiatkowski-Phillips-Schmidt-Shin (KPSS) Test

Augmented Dickey-Fuller (ADF) Test or Unit Root Test

The ADF test is the most popular statistical test. It is done with the following assumptions:

- Null Hypothesis (H0): Series is non-stationary
- Alternate Hypothesis (HA): Series is stationary
 - p-value > 0.05 Fail to reject (H0)
 - p-value ≤ 0.05 Accept (H1)

Kwiatkowski-Phillips-Schmidt-Shin (KPSS) Test

These tests are used for testing a NULL Hypothesis (H0) that will perceive the time series as stationary around a deterministic trend against the alternative of a unit root. Since TSA is looking for Stationary Data for its further analysis, we have to ensure that the dataset is stationary.

Model Selection and Boosting

What Is Model Selection

Model selection is the process of selecting one final machine learning model from among a collection of candidate machine learning models for a training dataset.

Model selection is a process that can be applied both across different types of models (e.g., logistic regression, SVM, KNN, etc.) and across models of the same type configured with different model hyperparameters (e.g., different kernels in an SVM).

When we have a variety of models of different complexity (e.g., linear or logistic regression models with different degree polynomials, or KNN classifiers with different values of K), how should we pick the right one?

For example, we may have a dataset for which we are interested in developing a classification or regression predictive model. We do not know beforehand as to which model will perform best on this problem, as it is unknowable. Therefore, we fit and evaluate a suite of different models on the problem.

Model selection is the process of choosing one of the models as the final model that addresses the problem.

Model selection is different from **model assessment**.

For example, we evaluate or assess candidate models in order to choose the best one, and this is model selection. Whereas once a model is chosen, it can be evaluated in order to communicate how well it is expected to perform in general; this is model assessment.

Need for Model Selection

Model selection refers to choose the best statistical machine learning model for a particular problem. For this task we need to compare the relative performance between models. Therefore, the loss function and the metric that represent it, becomes fundamental for selecting the right and non-overfitted model.

We can state a machine learning supervised problem with the following equation:

$$y = f(x) + \epsilon$$

This equation is composed with the x matrix that contains the predictor's factors $x_1, x_2, x_3, \dots, x_n$. These factors can be the lagged prices/returns of a time series or some others factors such as volume, foreign exchange rates, etc. y is the response vector that depend of the function f and the predictors x .

f contains the underlying relationship between the x features and the y response and can be modelled with a linear regression if the underlying relationship is linear or with a Random Forest or Support Vector Machine algorithm if the underlying relationship is non-linear.

E represent the error term, which is often assumed to have mean zero and a standard deviation of one.

Once we fit a particular model for a certain dataset, we need to define the loss function that we will use to assess model performance. Many measures can be used for the loss function. Some common measures for the loss function are the Absolute Error and the Squared Error between predicted values and real values.

Loss function as Absolute Error : $|y - \hat{y}|$

Loss function as Squared Error : $(y - \hat{y})^2$

Both choices are non-negative, so the best value for the loss function is zero. The Absolute Error and Squared Error above, compute the difference between the true value (y) and the prediction ($\hat{y}$) for each observation of the dataset.

Both the Absolute Error and Squared Error are vectors or arrays of $n \times 1$ dimension, reflecting the error term per each of the observations. In order to aggregate the error term of a certain model between all the predicted and real values of a variable, a popular measure is the Mean Squared Error which is simply the average of the squared loss:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where n is the number of observations

Boosting

What is Boosting?

Definition: The term ‘Boosting’ refers to a family of algorithms which converts weak learner to strong learners.

Let’s understand this definition in detail by solving a problem of spam email identification:

How would you classify an email as SPAM or not? Like everyone else, our initial approach would be to identify ‘spam’ and ‘not spam’ emails using following criteria. If:

1. Email has only one image file (promotional image), It’s a SPAM
2. Email has only link(s), It’s a SPAM
3. Email body consist of sentence like “You won a prize money of \$ xxxxxx”, It’s a SPAM
4. Email from our official domain “Analyticsvidhya.com”, Not a SPAM
5. Email from known source, not a SPAM

Above, we’ve defined multiple rules to classify an email into ‘spam’ or ‘not spam’. But, do you think these rules individually are strong enough to successfully classify an email? No.

Individually, these rules are not powerful enough to classify an email into ‘spam’ or ‘not spam’. Therefore, these rules are called as **weak learner**.

To convert weak learner to strong learner, we'll combine the prediction of each weak learner using methods like:

- Using average/ weighted average
- Considering prediction has higher vote

For example: Above, we have defined 5 weak learners. Out of these 5, 3 are voted as 'SPAM' and 2 are voted as 'Not a SPAM'. In this case, by default, we'll consider an email as SPAM because we have higher(3) vote for 'SPAM'.

How Boosting Algorithms works?

Now we know that, boosting combines weak learner a.k.a. base learner to form a strong rule. An immediate question which should pop in your mind is, '*How boosting identify weak rules?*'

To find weak rule, we apply base learning (ML) algorithms with a different distribution. Each time base learning algorithm is applied, it generates a new weak prediction rule. This is an iterative process. After many iterations, the boosting algorithm combines these weak rules into a single strong prediction rule.

Here's another question which might haunt you, '*How do we choose different distribution for each round?*'

For choosing the right distribution, here are the following steps:

Step 1: The base learner takes all the distributions and assign equal weight or attention to each observation.

Step 2: If there is any prediction error caused by first base learning algorithm, then we pay higher attention to observations having prediction error. Then, we apply the next base learning algorithm.

Step 3: Iterate Step 2 till the limit of base learning algorithm is reached or higher accuracy is achieved.

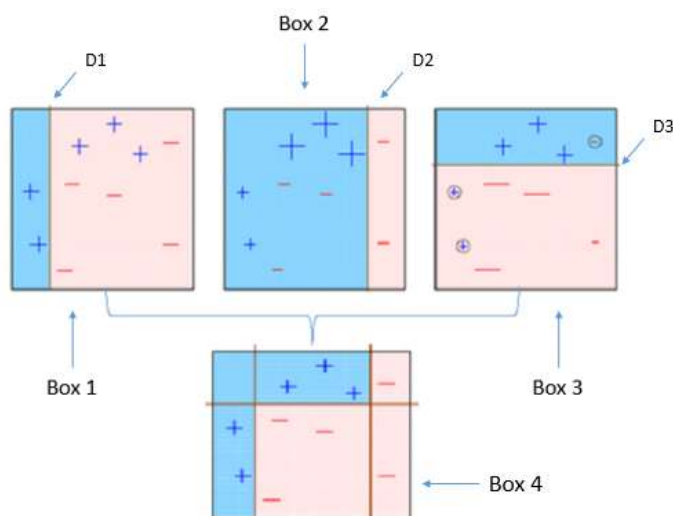
Finally, it combines the outputs from weak learner and creates a strong learner which eventually improves the prediction power of the model. Boosting pays higher focus on examples which are mis-classified or have higher errors by preceding weak rules.

Types of Boosting Algorithms

Underlying engine used for boosting algorithms can be anything. It can be decision stamp, margin-maximizing classification algorithm etc. There are many boosting algorithms which use other types of engine such as:

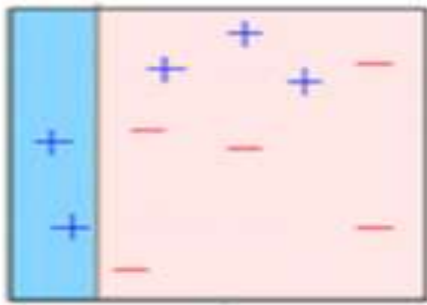
1. AdaBoost (**A**daptive **B**oosting)
2. Gradient Tree Boosting
3. XGBoost

Boosting Algorithm: AdaBoost

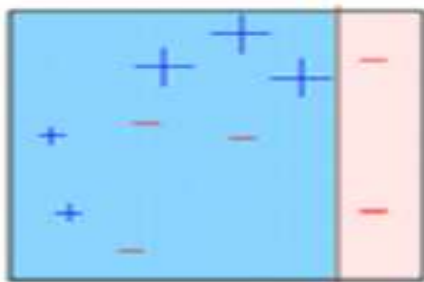


This diagram aptly explains Ada-boost. Let's understand it closely:

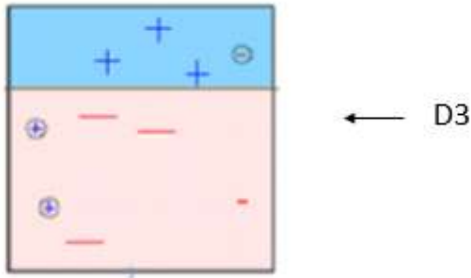
Box 1: You can see that we have assigned equal weights to each data point and applied a decision stump to classify them as + (plus) or - (minus). The decision stump (D1) has generated vertical line at left side to classify the data points. We see that, this vertical line has incorrectly predicted three + (plus) as - (minus). In such case, we'll assign higher weights to these three + (plus) and apply another decision stump.



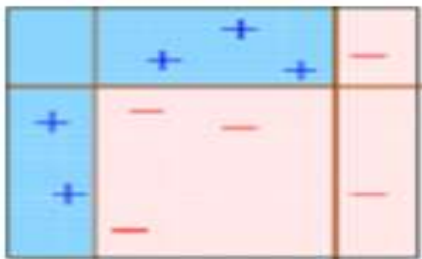
Box 2: Here, you can see that the size of three incorrectly predicted + (plus) is bigger as compared to rest of the data points. In this case, the second decision stump (D2) will try to predict them correctly. Now, a vertical line (D2) at right side of this box has classified three mis-classified + (plus) correctly. But again, it has caused mis-classification errors. This time with three -(minus). Again, we will assign higher weight to three - (minus) and apply another decision stump.



Box 3: Here, three – (minus) are given higher weights. A decision stump (D3) is applied to predict these mis-classified observation correctly. This time a horizontal line is generated to classify + (plus) and – (minus) based on higher weight of mis-classified observation.



Box 4: Here, we have combined D1, D2 and D3 to form a strong prediction having complex rule as compared to individual weak learner. You can see that this algorithm has classified these observation quite well as compared to any of individual weak learner.



AdaBoost (Adaptive Boosting) : It works on similar method as discussed above. It fits a sequence of weak learners on different weighted training data. It starts by predicting original data set and gives equal weight to each observation. If prediction is incorrect using the first learner, then it gives higher weight to observation which have been predicted incorrectly. Being an iterative process, it

continues to add learner(s) until a limit is reached in the number of models or accuracy.

Mostly, we use decision stamps with AdaBoost. But we can use any machine learning algorithms as base learner if it accepts weight on training data set. We can use AdaBoost algorithms for both classification and regression problem.